



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

MESTERSÉGES INTELLIGENCIA TANSZÉK

KCMamba: Kalman-vezérelt szelektív
állapottér-modell robusztus
idősor-előrejelzésre

Témavezető:

Kristian Fenech Arian
oktató

Szerző:

Kovács András (K2WEMI)
programtervező informatikus BSc

Budapest, 2026

EÖTVÖS LORÁND TUDOMÁNYEGYETEM
INFORMATIKAI KAR

SZAKDOLGOZAT TÉMABEJELENTŐ

Hallgató adatai:

Név: Kovács András

Neptun kód: K2WEMI

Képzési adatok:

Szak: programtervező informatikus, alapképzés (BA/BSc/BProf)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Fenech Kristian Arian

munkahelyének neve, tanszéke: ELTE IK, Mesterséges Intelligencia Tanszék

munkahelyének címe: 1117, Budapest, Pázmány Péter sétány 1/C.

beosztás és iskolai végzettsége: Oktató

A szakdolgozat címe: Kalman-alapú állapotter-modelle fejlesztése idősoros becsléshez

A szakdolgozat témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben szakdolgozat témájának leírását)

Motiváció

Számos alkalmazási terület – járműkövetés, robot szenzorfüzió, szenzorhálózatok és energia/pénzügyi előrejelzés – zajos, nagyméretű idősorok valós idejű feldolgozását igényli. A gyakorlatban alkalmazott módszerek gyakran számítási szempontból drágák, érzékenyek a mérési anomáliákra, és nehezen skálázhatók hosszú sorozatokra és magas állapotdimenziókra. A szakdolgozat célja egy Kalman-szűrőn alapuló állapotter-modelle megtervezése és kiértékelése.

Célkitűzések

1. Kalman-alapú modelle megtervezése SSM vagy Mamba modellek alapján.
2. Működés kiértékelése ilyen feltételek mellett.
3. Az új megközelítés empirikus összehasonlítása a standard alapvonallakkal több, eltérő adathalmazon.

Kutatási kérdések

- A bizonytalanságbecslés közvetlenül iterálható-e egy strukturált állapotter-modellebe, és milyen mértékben előnyös ez?
- Hogyan viselkedik a modelle különböző zajszintek, részleges szenzorhibák és kiugró értékek esetén?
- Mennyire általánosítható a megközelítés különböző területeken (követés, szenzorfüzió, pénzügyi/energia idősorok)?

Módszertani megközelítés

A szakdolgozat egy Kalman-családba tartozó, állapotter-becslőt épít.

A fejlesztés a Kalman-szűrőben található bizonytalanságbecslés és predikció-frissítés hurkok integrálására összpontosít a Strukturált Állapotter-modellek központi blokkjába.

Értékelési terv és metrikák

A modelle több adathalmazon tipikus alapértékekkel hasonlítjuk össze. A főbb metrikák közé tartozik az RMSE/MAE (becslési hiba), a negatív log-valószínűség vagy a kalibrációs metrikák (bizonytalanságkezelés). A dedikált ablációs vizsgálatok számszerűsítik, hogy az egyes komponensek hogyan járulnak hozzá a sebesség- és stabilitásnövekedéshez.

Budapest, 2025. 10. 15.

Kivonat

Ez a szakdolgozat a **KCMamba** (Kalman Core-Attention Mamba) architektúrát mutatja be, amely differenciálható Kalman-szűrési mechanizmust integrál a Mamba szelektív állapottér-modellbe (SSM) a zajrobosztus idősor-előrejelzés érdekében. A javasolt architektúra tanult Q_{net} -, R_{net} - és K_{net} -alhálózatokat alkalmaz, amelyek adaptívan szabályozzák a mérési és folyamat zaj arányát, ezáltal a modell zajmentes adatokon passzív előrejelzőként, zajosított bemeneten aktív szűrőként viselkedik.

Empirikus értékelést végzünk három nyilvános LTSF-adatkészleten (Exchange-Rate, ETTh1, ETTh2), szintetikus zajosított ($\sigma \in \{0, 1, 3, 5\}$) és alacsony adatbőségű ($\text{stride} \in \{1, 16\}$) körülmények között. Kísérleteink szerint a KCMamba MSE-degradációja $\sigma=0$ -ról $\sigma=5$ -re átlagosan 1 528%, míg a Fair Mamba baseline esetén 28 234%, ez közel 18,5×-es robusztusság-előnyt jelent.

A szakdolgozat a teljes szoftverrendszer leírását is tartalmazza: egy aszinkron, eseményvezérelt kereskedési motort hexagonális architektúrában, webalapú irányítópultot, visszatesztelő keretrendszert és élő Binance-kereskedési adaptert.

Abstract

This thesis presents **KCMamba** (Kalman Core-Attention Mamba), a hybrid neural architecture that integrates a differentiable Kalman filtering mechanism into the Mamba selective state-space model (SSM) for noise-robust time series forecasting. The proposed architecture employs learned Q_{net} , R_{net} , and K_{net} subnetworks that adaptively regulate the ratio of process and measurement noise, enabling the model to behave as a passive predictor on clean data and as an active filter under noisy conditions.

We conduct empirical evaluation on three public LTSF benchmarks (Exchange-Rate, ETTh1, ETTh2) under synthetically injected noise ($\sigma \in \{0, 1, 3, 5\}$) and low data-availability regimes (stride $\in \{1, 16\}$). Our results show that KCMamba achieves an average MSE degradation of 1 528% as noise increases from $\sigma=0$ to $\sigma=5$, compared to 28 234% for Fair Mamba, a $18.5\times$ robustness advantage.

The thesis also describes the complete production software system: an asynchronous, event-driven trading engine built in a hexagonal architecture, a web-based dashboard, a backtesting framework, and a live Binance trading adapter.

Tartalomjegyzék

Kivonat	2
Abstract	3
1. Bevezetés	7
1.1. Motiváció	7
1.2. Célkitűzések	8
1.3. Hozzájárulások	8
1.4. Dolgozat felépítése	9
2. Irodalmi háttér	10
2.1. Kalman-szűrő és általánosításai	10
2.1.1. Tanult Kalman-szűrők	11
2.2. Állapottér-modellek	11
2.2.1. S4	11
2.2.2. Mamba	12
2.2.3. Fair Mamba	12
2.3. Hosszú távú idősor-előrejelzés	13
2.3.1. Problémaformulázás	13
2.3.2. Referenciamodellek	13
2.3.3. Adatkészletek	14
2.4. Zajrobustusság	14
3. A KCMamba architektúra	16
3.1. Áttekintés	16
3.2. KCMamba blokk	18
3.2.1. Bemeneti projekció és 1D-konvolúció	18
3.2.2. Kalman-paraméter hálózatok	19
3.2.3. Kalman-gain számítás	20

3.2.4.	Állapotfrissítés párhuzamos scan-nel	20
3.2.5.	Összegzés és reziduál kapcsolat	21
3.3.	KCMamba verem	22
3.4.	Paraméterkomplexitás	25
3.5.	Összehasonlítás a standard Mamba-val	26
3.6.	Rendszerarchitektúra	27
3.7.	Modul- és osztályszerkezet	27
4.	Kísérletek és értékelés	29
4.1.	Kísérleti beállítások	29
4.1.1.	Hardver és szoftver	29
4.1.2.	Adatkészletek	29
4.1.3.	Zajmodell	29
4.1.4.	Modellkonfiguráció és tanítási hiperparaméterek	30
4.2.	Eredmények	30
4.2.1.	Exchange-Rate adatkészlet	30
4.2.2.	ETTh1 adatkészlet	31
4.2.3.	ETTh2 adatkészlet	32
4.2.4.	Összefoglaló statisztika	32
4.3.	A Kalman-gain dinamikája	33
4.4.	Diszkusszió	36
4.5.	Hosszú szekvenciák vizsgálata	37
4.6.	Stride $\times$ Hosszú Szekvencia	38
5.	Fejlesztői dokumentáció	41
5.1.	Megvalósítási technológiák	41
5.2.	Komponens-terv	42
5.3.	A webalapú irányítópult	42
5.3.1.	Főoldal, Áttekintő nézet	43
5.3.2.	Kereskedési jelzések	43
5.3.3.	Modell diagnosztika	44
5.3.4.	Kalman-szűrő diagnosztika	44
5.3.5.	Offline tanítás	44
5.3.6.	Benchmark és modell-összehasonlítás	45
5.4.	Tesztelés	45

5.5. A visszatesztelő keretrendszer	47
5.6. Kockázatkezelés	47
5.7. Élő kereskedés Binance-szal	48
5.8. A rendszer futtatása	48
6. Felhasználói dokumentáció	49
6.1. Használati esetek	49
6.2. A szoftver célja és célközönsége	50
6.3. Felhasznált módszerek áttekintése	50
6.4. Rendszerkövetelmények	50
6.5. Telepítési útmutató	51
6.6. Használati útmutató	51
6.7. Hibaelhárítás	53
6.8. Futásidei üzenetek és hibaelhárítás	53
7. Összegzés	55
7.1. Főbb eredmények	55
7.2. Korlátok	56
7.3. Jövőbeli irányok	57
Köszönetnyilvánítás	58
Irodalomjegyzék	58
Ábrajegyzék	61

1. fejezet

Bevezetés

Az idősor-előrejelzés a gépi tanulás egyik kiterjedten kutatott részterülete, amelynek gyakorlati jelentőségét a valós adatok szinte kivétel nélkül jelen lévő zajossága adja. Szenzorkiesések, aggregációs torzítások és piaci mikrostruktúra-zaj egyaránt rontják a jel/zaj arányt, amelyre a jelenlegi modellek érzékenynek bizonyulnak. A hosszú távú idősor-előrejelzés (LTSF) feladata ennél is nehezebb. Adott L hosszúságú előzmény alapján kell becsülni a következő H lépést, ahol H nagyságrendileg elérheti vagy meghaladhatja L -et.

1.1. Motiváció

A modern szekvencia-model-családok, a Mamba-alapú szelektív állapottér-modellek [1, 2] és a klasszikus rekurrens hálózatok [3] az adatminőségi problémát közvetett általánosítás révén kezelik, vagyis a zaj- és jelkomponens szétválasztását kizárólag az adatvezérelt reprezentációtanulásra bízzák. Zajmentes adatokon ez az eljárás kielégítően teljesít, ám a megnövelt zajszintre való általánosítóképesség a mérések alapján nem garantált.

A Kalman-szűrő [4] ezzel szemben *explicit* statisztikai modellt állít fel a folyamat-zajra (Q) és a mérési zajra (R), és ezek arányából származtat egy K gain együtthatót, amely az előzmény-állapot és az aktuális megfigyelés közötti optimális súlyozást határozza meg. Ennek az induktív torzításnak egy differenciálható neurális hálózatba történő beépítése elméleti szempontból ígéretes megközelítés a zajrobustság növelésére.

Ezen megfontolásra épül a jelen szakdolgozatban bemutatott **KCMamba** architektúra.

1.2. Célkitűzések

Három kérdést vizsgálunk:

1. Versenyképes-e a KCMamba tiszta adatokon a standard LTSF modellekkel szemben?
2. Mekkora MSE-degradációt szenvednek el a modellek szintetikus zajosított adatokon ($\sigma \in \{0, 1, 3, 5\}$)?
3. Kevés tréning-ablak ($\text{stride} \in \{1, 16\}$) esetén melyik architektúra generalizál jobban?

1.3. Hozzájárulások

- A **KCMamba** architektúra bemutatása, amely tanult Q_{net} , R_{net} és K_{net} alhálózatokat ágyaz be egy differenciálható Kalman-szűrőbe, és a rekurzió kiértékelését párhuzamos prefix-scan alapú GPU-implementációval gyorsítja.
- Mérések arról, hogy a tanult K valóban adaptívan változik a zajszinttel: $\sigma = 0$ -nál $K \approx 0,71$ (passzív átengedés), $\sigma = 5$ -nél $K \approx 0,07$ (erős szűrés).
- Összehasonlítás három adatkészleten (Exchange-Rate, ETTh1, ETTh2): a Fair Mamba átlag 28 234% MSE-degradációt mutat zajjal, a KCMamba 1 528%-ot ($18,5\times$ kisebb romlás).
- Hosszú szekvencia vizsgálat (ETTm1, Exchange, $T \in \{96, 192, 336, 512\}$,): a KCMamba $T = 336$, $\sigma = 1$ konfigurációban **31,7%**-kal jobb MSE-t ér el, mint az LSTM (0,502 vs 0,735). $T = 512$, $\sigma = 3$ -nál a KCMamba 0,773, a Fair Mamba 2,121, vagyis a KCMamba 63,6%-kal jobb. $\sigma = 3$ -nál az ETTm1-en a KCMamba MSE monoton csökken $T = 96$ -tól $T = 512$ -ig ($0,673 \rightarrow 0,466$, -31%).
- Stride $\times$ szekvenciahossz vizsgálat (Exchange, $s \in \{1, 16\}$, $T \in \{192, 336, 512\}$): $s=1$, $\sigma=3$ -nál a KCMamba $T=512$ -nél **+35%**-kal jobb az

LSTM-nél (0,836 vs 1,292). Az előny a szekvenciahossz növekedésével monoton nő. $s=16$, $\sigma \geq 3$ -nál a Fair Mamba MSE-je 8,1–8,5-re ugrik amíg a KCMamba 1,1–1,3 között marad.

- Nyílt forráskódú megvalósítás (Python/PyTorch/CUDA) és egy aszinkron kereskedési rendszer a modell éles környezetbe helyezéséhez.

1.4. Dolgozat felépítése

A 2. fejezet összefoglalja a Kalman-szűrő, az SSM-ek és az LTSF-kutatás idevágó irodalmát. A 3. fejezet ismerteti a KCMamba architektúrát és matematikai hátterét. A 4. fejezetben a kísérleti beállítás és az eredmények szerepelnek. A 5. fejezet a megvalósított szoftverrendszert tárgyalja. Az 7. fejezet levonja a következtetéseket.

2. fejezet

Irodalmi háttér

2.1. Kalman-szűrő és általánosításai

A Kalman-szűrő [4] minimális varianciájú, torzítatlan, lineáris becslő olyan rendszerekre, amelyek $x_t \in \mathbb{R}^n$ állapota Gauss-zajjal perturbált lineáris dinamikai modell szerint fejlődik:

$$x_t = Fx_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q), \quad (2.1)$$

$$y_t = Hx_t + v_t, \quad v_t \sim \mathcal{N}(0, R), \quad (2.2)$$

ahol F az állapotátmeneti mátrix, H a mérési mátrix, $Q \in \mathbb{R}^{n \times n}$ a folyamatzaj-kovariancia, és $R \in \mathbb{R}^{m \times m}$ a mérési-zaj-kovariancia. A szűrő két felváltva ismétlődő fázisban működik.

Az *előrejelzési lépésben* (időfrissítés), mielőtt a y_t mérés megérkezne, a szűrő az állapotbecslést és a hibakovarianciát előrevetíti:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1}, \quad (2.3)$$

$$P_{t|t-1} = FP_{t-1|t-1}F^\top + Q, \quad (2.4)$$

ahol $P_{t|t-1} \in \mathbb{R}^{n \times n}$ az előrejelzett hibakovariancia. Nagy $P_{t|t-1}$ értéke magas előrejelzési bizonytalanságot jelez. Nagy Q azt jelenti, hogy az állapot gyorsan és kiszámíthatatlanul változik.

A *korrekciós lépésben* (mérésfrissítés), amint a y_t megfigyelés megérkezik, a szűrő az *innováció* $e_t = y_t - H\hat{x}_{t|t-1}$ alapján korrigálja becslését, a Kalman-gain K_t által

súlyozva:

$$K_t = P_{t|t-1} H^\top (H P_{t|t-1} H^\top + R)^{-1}, \quad (2.5)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t e_t, \quad (2.6)$$

$$P_{t|t} = (I - K_t H) P_{t|t-1}. \quad (2.7)$$

A (2.5). egyenlet a zaj-jel-egyensúly alapvető összefüggését tükrözi. Ha az R mérési zaj nagy az előrejelzett $H P_{t|t-1} H^\top$ kovarianciához képest, a nevező dominál és $K_t \rightarrow 0$: a szűrő nem bíz a mérésben, és az előzmény-állapotra támaszkodik. Ha $R \rightarrow 0$, akkor $K_t \rightarrow H^{-1}$, és a megfigyelés majdnem teljesen felülírja az állapotot.

A skaláris esetben ($F = H = 1$) a gain egyszerűsödik:

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R}. \quad (2.8)$$

Stacionárius feltételek mellett a $P_{t|t-1}$ hibakovariancia egy fix $\bar{P}$ értékre konvergál, és az egyensúlyi gain kizárólag a Q/R aránytól függ: magas folyamat zaj (Q) a K -t 1 felé tolja (bízunk a mérésben), alacsony Q/R arány K -t 0 felé tolja (bízunk a modellben). (3.2.3. szakasz).

2.1.1. Tanult Kalman-szűrők

A neurális Kalman-irodalomban a F, H, Q, R mátrixokat neurális hálózatokkal paraméterezik [5, 6]. A *Deep Kalman Filter* [5] nemlineáris átmenetet enged meg, a *Kalman Variational Autoencoder* [6] a latens állapotok szétválasztott reprezentációját tanítja. A KCMamba ezeken az elveken épít, de nem autoencoderként, hanem szekvencia-előrejelzőként.

2.2. Állapottér-modellek

2.2.1. S4

Az S4 (Structured State Spaces for Sequences) [1] egy folytonos-idejű állapottér-modellből indul ki, amelyben egy d -dimenziós bemeneti sorozatot egy N -dimenziós

látens dinamikán keresztül modellez:

$$h'(t) = Ah(t) + Bu(t), \quad (2.9)$$

$$y(t) = Ch(t), \quad (2.10)$$

ahol $A \in \mathbb{R}^{N \times N}$ az állapotátmeneti mátrix, $B \in \mathbb{R}^{N \times d}$ a bemeneti projekció, $C \in \mathbb{R}^{d \times N}$ a kimeneti projekció, és $h(t) \in \mathbb{R}^N$ a látens állapot. Diszkrétizálva egy Δ időlépéssel:

$$h_t = \bar{A} h_{t-1} + \bar{B} u_t, \quad y_t = C h_t, \quad (2.11)$$

ahol $\bar{A} = e^{\Delta A}$ és $\bar{B} \approx (\Delta A)^{-1}(e^{\Delta A} - I)\Delta B$. Az S4 kulcsinnovációja az A mátrix HiPPO-inicializációja, amely az állapotnak polinomiális bázisú hosszú távú memóriát biztosít. A konvolúciós nézet, $\bar{K} = (C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^{T-1}\bar{B})$ kernel, lehetővé teszi, hogy a tanítás $O(T \log T)$ -ben gyors Fourier-transzformációval (FFT) végezhető, míg az inferencia a rekurrens formában $O(1)$ memóriával fut lépésenként.

2.2.2. Mamba

A Mamba [2] az S4 alapjaira építve bevezeti az *input-függő szelektivitást*: míg az S4-ben az A , B , C mátrixok rögzítettek a teljes szekvenciára, a Mamba ezeket az aktuális bemenettől függővé teszi. Ez a szelektivitás a lényege: a modell minden időlépésben eldönti, mit őrizzen meg az állapotban és mit felejtson el. A bemeneti projekciós mátrix $B(u_t) \in \mathbb{R}^{N \times d}$, a kimeneti projekció $C(u_t) \in \mathbb{R}^{d \times N}$ és az időléptékező $\Delta(u_t) \in \mathbb{R}^d$ mindegyike az aktuális $u_t \in \mathbb{R}^d$ bemeneti vektortól függ:

$$h_t = \bar{A}(u_t) h_{t-1} + \bar{B}(u_t) u_t, \quad y_t = C(u_t) h_t, \quad (2.12)$$

ahol $h_t \in \mathbb{R}^N$ a látens állapotvektor (N az állapotméret), $\bar{A}(u_t) = e^{\Delta(u_t)A}$ és $\bar{B}(u_t) \approx \Delta(u_t)B$ a Δ alapján diszkrétizált mátrixok. A modell így szelektíven dönti el, hogy a bemeneti sorozat melyik részét tárolja tartósan az állapotban, és melyiket hagyja figyelmen kívül. Hardver-barát párhuzamos prefix-scan implementációval $O(T)$ memória- és időkomplexitást ér el.

2.2.3. Fair Mamba

Az összehasonlítási alap, amelyet *Fair Mamba* névvel illetünk, egy csatornánkénti (*channel-wise*) Mamba, amelyet a KCMamba-éval azonos hiperparaméterekkel és

tanítási protokoll szerint tanítunk. A *fair* jelző arra utal, hogy a paraméterbudget és az adatmennyiség azonos, így az eredménybeli különbség kizárólag az architektúrából fakad.

2.3. Hosszú távú idősor-előrejelzés

2.3.1. Problémaformulázás

A hosszú távú idősor-előrejelzés (LTSF) egy $\mathbf{X} = (x_1, \dots, x_L) \in \mathbb{R}^{L \times d}$ bemeneti ablakot tekint, amelynek L historikus lépése d csatornán van megadva, és $\hat{\mathbf{X}} = (\hat{x}_{L+1}, \dots, \hat{x}_{L+H}) \in \mathbb{R}^{H \times d}$ előrejelzését kívánja adni a következő H lépésre. A cél a normált átlagos négyzetes hiba minimalizálása:

$$\text{MSE} = \frac{1}{Hd} \sum_{h=1}^H \sum_{j=1}^d \frac{(x_{L+h,j} - \hat{x}_{L+h,j})^2}{\text{Var}(x_{\cdot,j})}, \quad (2.13)$$

ahol a csatornánkénti varianciával való normálás különböző skálájú adatkészletek eredményeit összehasonlíthatóvá teszi. A *hosszú távú* jelző a $H \geq L$, illetve H és L hasonló nagyságrendű esetét jelöli: az ilyen feladatban a modellnek a visszatekintési ablakán jócskán túlra kell előrejelezni, ez pedig lényegesen nehezebb, mint a rövid távú eset, ahol $H \ll L$.

2.3.2. Referenciamodellek

Informer [7] volt az első, kifejezetten hosszú távú előrejelzésre tervezett Transformer-változat. A standard önfigyelés $O(L^2)$ idő- és memóriakomplexitása hosszú bemeneti ablakoknál elfogadhatatlanul magas. Az *Informer* ezt *ProbSparse attention*-tel orvosolja: KL-divergencia-bebecslés alapján azonosítja az $O(\log L)$ leginformatívabb lekérdező pozíciót, ezzel $O(L \log L)$ -re csökkenti a komplexitást. Disztillációs művelet felezi a szekvenciahosszat az enkoderblokkok között, így több ezer lépéses visszatekintési ablakok is kezelhetők.

Autoformer [8] minden sorozatot egy trendkomponensre (mozgóátlag-kernellel becsülve) és egy szezonális maradéktagra bont, és a Transformatort kizárólag a maradéktagra alkalmazza. Pontszorzat-figyelem helyett *Auto-Korrelációs* mechanizmust használ: megkeresi azt a τ késleltetést, amely maximalizálja a lekérdező jel autokorrelációját, majd ezeken a késleltetési pozíciókon összesíti az értékeket. A

mechanizmus FFT-vel $O(L \log L)$ komplexitású, és periodikus jelekre illeszkedő induktív torzítással rendelkezik.

DLinear [9] elveti az architektúráis bonyolultságot, és az előrejelzést minden visszatekintési ablakra egyetlen lineáris réteggel állítja elő, opcionálisan trend-szezónális felbontás után. Egyszerűsége ellenére a *DLinear* számos Transformer-változattal azonos szinten teljesített, sőt azokat felülmúlta szabványos benchmarkokon, bizonyítva, hogy az architektúráis bonyolultság nem feltétele a jó előrejelzési teljesítménynek. Referenciamodellként releváns marad, mert minden új modellnek igazolnia kell, hogy a magasabb számítási ráfordítás indokolt ehhez az egyszerű alapmodellhez képest.

2.3.3. Adatkészletek

Az Exchange-Rate adatkészlet [10] nyolc valutaárfolyamot tartalmaz, amelyeket 1990 és 2016 között napi szinten vettek fel, csatornánként 7588 időlépéssel. Az árfolyam-adatok dinamikája majdnem egységgyök-jellegű: minden napi változás véletlenszerű bolyongáshoz közelít, szinte elhanyagolható vissza-átlagolással. Ez kemény feladatot jelent az autoregresszív modelleknek, és természetes tesztkörnyezetet jelöl a Kalman-struktúra számára, ahol a jel-zaj arány eleve alacsony.

Az ETTh1 és ETTh2 adatkészletek [7] kínai transzformátorállomásokból gyűjtöttek óránkénti villamosenergia-terhelést és olajhőmérsékletet két éven keresztül (17420 lépés, $d = 7$ csatorna). Az ETTh1 egyértelmű napi periodicitást mutat, az ETTh2 zajosabb és szabálytalanabb. A két adatkészlet eltérő nehézségű tesztkörnyezetet nyújt a zajrobosztusság kiértékeléséhez.

2.4. Zajrobosztusság

Az idősor-modellek zajérzékenységének szisztematikus elemzése az LTSF-irodalomban aláreprézantált terület. A PatchTST [11] patch-tokenizálással csökkenti a szomszédos mintavételezési torzítást. Ennek közvetett zajcsökkentő hatása van, bár nem ez az eljárás fő célkitűzése.

A jelen munkában az értékelési protokoll: $\tilde{x}_t = x_t + \varepsilon_t$, ahol $x_t \in \mathbb{R}^d$ az eredeti idősorjel, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2 I)$ additív Gauss-zaj ($\sigma > 0$ a szórás, $I \in \mathbb{R}^{d \times d}$ az egységmátrix), és $\sigma \in \{0, 1, 3, 5\}$ a vizsgált zajszintek halmaza. A tanító- és tesztalmazt egyaránt

azonos σ -val zajosítjuk, tehát nem domain-shift-ről, hanem egyenletes adatminőség-romlásról van szó.

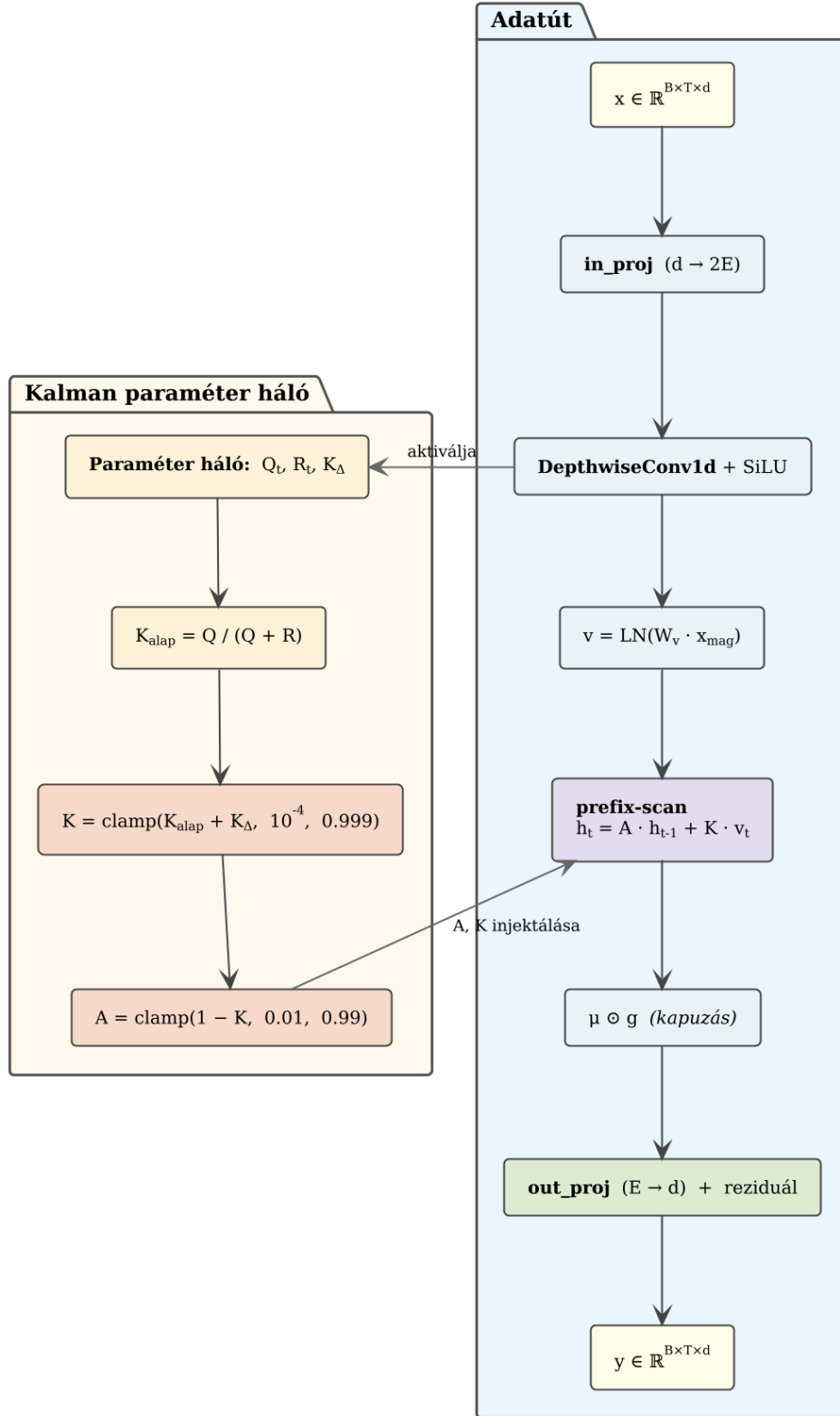
3. fejezet

A KCMamba architektúra

3.1. Áttekintés

A KCMamba (Kalman Core-Attention Mamba) egy hibrid neurális architektúra, amely a Mamba SSM [2] rekurrens dinamikájába egy differenciálható, tartalomfüggő Kalman-szűrő réteget integrál. A modell nem rögzített A felejtési faktorral dolgozik, hanem egy, a bemenetből tanult K -gainből származtatott $A = 1 - K$ együtthatót használ az állapotfrissítésben. Ez lehetővé teszi, hogy az előzmény- állapot és az aktuális megfigyelés közötti súlyozás a zajszintnek megfelelően, adaptívan álljon be.

A modell két szintből épül fel: az alap feldolgozó egység a `KCMambaBlock` (tartalom-függő Kalman-szűrés és állapotfrissítés), fölötte a `KCMambaStack`: N egymás utáni KC-blokk cross-attention réteggel a végén.



3.1. ábra. A KCMamba blokk teljes adatfolyam-diagramja. A sárga blokkok (Q_{net} , R_{net} , K_{net}) tanítják a Kalman-statisztikákat, a lila blokkban történik a párhuzamos prefix-scan GPU-n $O(\log T)$ szinkronizációval. A zöld blokkok a bemeneti és kimeneti projekciókat jelölik.

3.2. KCMamba blokk

Legyen a bemenő szekvencia $x \in \mathbb{R}^{B \times T \times d}$, ahol B a kötegméret, T a szekvencia-hossz, és d a jellemzők száma.

3.2.1. Bemeneti projekció és 1D-konvolúció

A bemenetet két részre vetítjük egy lineáris réteggel, majd SiLU aktivációs kapuzást alkalmazunk:

$$[x_{\text{cw}}, x_{\text{gate}}] = W_{\text{in}} x, \quad (3.1)$$

$$\text{gate} = \text{SiLU}(x_{\text{gate}}), \quad (3.2)$$

ahol $W_{\text{in}} \in \mathbb{R}^{2d_{\text{inner}} \times d}$ a bemeneti projekciós mátrix, d_{inner} a belső csatornadi-menzió, x_{cw} a Kalman-szűrési ághoz, x_{gate} a kapuzáshoz továbbított csatorna, és $\text{SiLU}(z) = z \cdot \sigma(z)$ a Sigmoid Linear Unit aktiváció, ahol $\sigma(z) = 1/(1 + e^{-z})$ a logisztikus szigmoid. A kétágú projekció két különböző szerepet tölt be. Az x_{cw} csatorna a Kalman-szűrési ág bemenete, míg az x_{gate} csatorna a kimenet amplitúdójának füg-getlen modulálásáért felel. Ez a szétválasztás megakadályozza, hogy a kapuzási jel beavatkozzon a Kalman-állapot dinamikájába. A SiLU aktiváció ReLU-val szemben azért előnyös, mert sima és kis negatív értékeket is megenged, ami a korai tanítás során javítja a gradiens-átmenetet.

A globális szekvenciális scan előtt az x_{cw} csatornán egy depthwise 1D-konvolúció ($k = 4$ kernel) biztosítja a lokális kontextus-integrációt:

$$x_{\text{core}} = \text{SiLU}(\text{Conv1D}(x_{\text{cw}})). \quad (3.3)$$

A konvolúció csatornánként egy szűrőt alkalmaz (depthwise), ezért mindössze $k \cdot d_{\text{inner}}$ extra paramétert igényel, miközben rövid-horizontú mintázatokat, például rövid-távú lendületet és trendirányokat, ragad meg, mielőtt a Kalman-szűrő a teljes szek-venciát feldolgozza.

3.2.2. Kalman-paraméter hálózatok

Három kis, kétrétegű, teljesen összekötött alhálózat tanítja meg a Kalman-szűrő statisztikáit. A Q_{net} és R_{net} hálózatok az időlépésenkénti zajvarianciákat becslik:

$$Q_t = \text{softplus}(Q_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(q_{\text{scale}}), \quad (3.4)$$

$$R_t = \text{softplus}(R_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(r_{\text{scale}}). \quad (3.5)$$

ahol $Q_t > 0$ a *folyamatzaj* becsült varianciája, vagyis az állapot-dinamikára vonatkozó bizonytalanság, $R_t > 0$ a *mérési zaj* becsült varianciája, vagyis a bemeneti megfigyelés bizonytalansága, $q_{\text{scale}}, r_{\text{scale}} \in \mathbb{R}$ tanítható skalár paraméterek, és $\text{softplus}(z) = \ln(1 + e^z) > 0$ garantálja a pozitivitást. A $q_{\text{scale}} = -2,0$, $r_{\text{scale}} = 1,5$ inicializációk biztosítják, hogy kezdetben $R \gg Q$, vagyis $K_{\text{base}} \approx 0.07$, ami erős szűrési módot jelent alapértelmezésként.

Egy harmadik alhálózat, a K_{net} , szintén kétrétegű, teljesen összekötött Multi-Layer Perceptron, és közvetlen reziduál-korrekción nyújt a végső gainhez, ahogy azt a 3.2.3. szakasz részletezi. A $Q_{\text{net}}/R_{\text{net}}$ pár a lassan változó zajkörnyezetet modellezi, míg a K_{net} azonnali reakcióra képes gyors, tartalomfüggő eseményekre, például hirtelen volatilitásnövekedésre vagy trendfordulóra, amelyekre a zaj-arány becslése még nem alkalmazkodott.

3.2.3. Kalman-gain számítás

Az alap Kalman-gain a folyamat- és mérési zaj *arányából* következik, ami a becslélméleti steady-state Kalman-gain klasszikus képlete:

$$K_{\text{base},t} = \frac{Q_t}{Q_t + R_t + \varepsilon}, \quad (3.6)$$

ahol $\varepsilon = 10^{-6}$ a nullával való osztást megelőző numerikus stabilitási konstans. Ha $Q \gg R$, azaz az állapot gyorsan változik, akkor $K \rightarrow 1$, és a modell szorosan követi az aktuális megfigyelést. Ha $R \gg Q$, azaz a mérés zajos, akkor $K \rightarrow 0$, és az előző állapot dominál. Ez egy fizikailag értelmes, kontrollélméleti elvekre támaszkodó prior.

Ehhez járul egy tartalom-függő korrekció a K_{net} alhálózattól, ami olyan gyors rezsinváltások kezelésére képes, amelyekre a lassan alkalmazkodó Q/R arány még nem reagált:

$$K_t = \text{clamp}\left(K_{\text{base},t} + 0.3(\sigma_{\text{sig}}(K_{\text{net}}(x_{\text{core}})) - 0.5), [10^{-4}, 0.999]\right), \quad (3.7)$$

ahol $\sigma_{\text{sig}}(\cdot) = 1/(1 + e^{-(\cdot)})$ a logisztikus szigmoid függvény, amelynek értékkészlete a $(0, 1)$ nyílt intervallum, a -0.5 eltolás nullára centrálja a korrekciót inicializáláskor, és a 0.3 szorzó $(-0.15, +0.15)$ -re korlátozza a hatókörét. A $[10^{-4}, 0.999]$ -re való vágás két degenerált esetet zár ki. $K_t \approx 0$ befagyasztaná az állapotfrissítést, mivel nem integrálna új információt. $K_t \approx 1$ esetén a modell az előző lépés teljes memóriáját eldobná.

3.2.4. Állapotfrissítés párhuzamos scan-nel

Az $A_t = \text{clamp}(1 - K_t, 0.01, 0.99)$ felejtési faktorról a diszkrét Kalman-rekurzió:

$$\begin{aligned} h_t &= A_t \odot h_{t-1} + K_t \odot v_t, \\ v_t &= \text{LayerNorm}(W_v x_{\text{core},t}), \end{aligned} \quad (3.8)$$

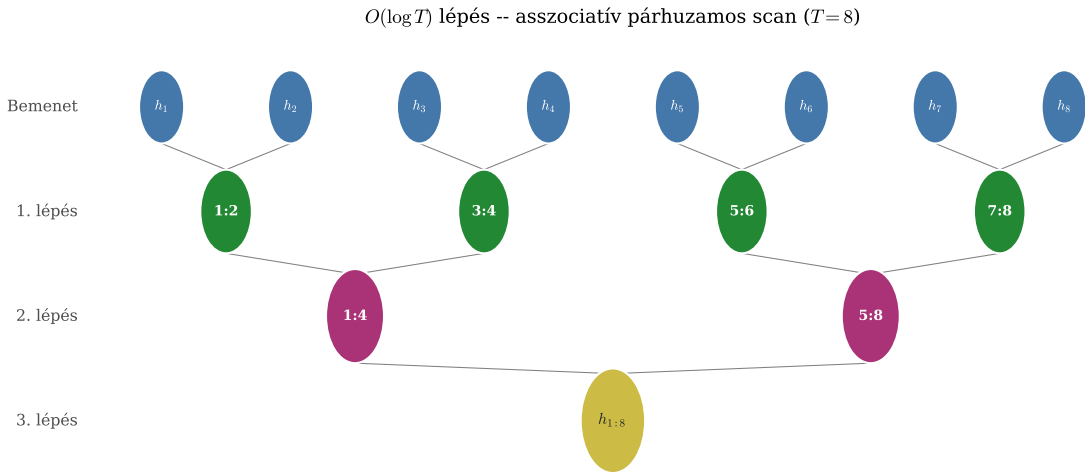
ahol $A_t \in (0, 1)^{d_{\text{inner}}}$ a felejtési faktor vektora, amelynél $A_t \rightarrow 1$ erős memóriát, $A_t \rightarrow 0$ erős frissítést jelent, $\odot$ az elem-szintű (Hadamard) szorzás, $h_{t-1} \in \mathbb{R}^{d_{\text{inner}}}$ az előző lépés látens állapota, v_t az aktuális értékvektor, és $W_v \in \mathbb{R}^{d_{\text{inner}} \times d_{\text{inner}}}$ tanítható projekció. A rekurzió hatékony kiértékelése a Blelloch-féle párhuzamos prefix-scan

algoritmussal [12] valósul meg:

$$A_{\text{new}}[t] \leftarrow A[t] \cdot A[t - \text{step}], \quad (3.9)$$

$$B_{\text{new}}[t] \leftarrow B[t] + A_{\text{orig}}[t] \cdot B[t - \text{step}], \quad (3.10)$$

ahol $A[t]$ és $B[t]$ az asszociatív prefix-pár (a_t, b_t) tagjai, a_t a halmozódó felejtési faktor, b_t a halmozódó bemenet-összeg. Ezek *nem* azonosak az SSM A, B mátrixaival, és a lépésköz $\text{step} \in \{1, 2, 4, \dots, T/2\}$ duplázódik. A rekurzió $O(\log T)$ szinkronizációs lépéssel párhuzamosítható GPU-n.



3.2. ábra. A Blelloch-féle párhuzamos prefix-scan vizualizációja 8 elemű szekvencián. Minden lépésben a step -nyi távolságú két elem kombinálódik. $\log_2 T = 3$ lépés után minden h_t tartalmazza az előzmény-összesítést. Ez teszi lehetővé a szekvenciális állapotfrissítés hatékony GPU-s párhuzamosítását.

3.2.5. Összegzés és reziduál kapcsolat

A Kalman-állapot-sorozat és a kapuzás kombinálja a kimenetet:

$$y_{\text{KC}} = W_{\text{out}}(\mu \odot \text{gate}), \quad (3.11)$$

ahol $\mu = (h_1, \dots, h_T) \in \mathbb{R}^{T \times d_{\text{inner}}}$ az összes időlépés Kalman-állapota, $W_{\text{out}} \in \mathbb{R}^{d \times d_{\text{inner}}}$ a kimeneti projekció, és $\odot$ az elem-szintű szorzás. Egy skalárkapuzással vezérelt reziduál kapcsolat biztosítja a gradiens átmenő útját:

$$\text{out} = y_{\text{KC}} + \sigma(b_{\text{res}}) \cdot (W_{\text{res}} x - y_{\text{KC}}), \quad (3.12)$$

ahol $b_{\text{res}} \in \mathbb{R}$ tanítható skalár, amelyet $-1,0$ -ra inicializálunk, így $\sigma(b_{\text{res}}) \approx 0,27$, W_{res} a reziduál projekció, és a kis $\sigma(b_{\text{res}})$ alaphelyzetben az SSM-ágot részesíti előnyben.

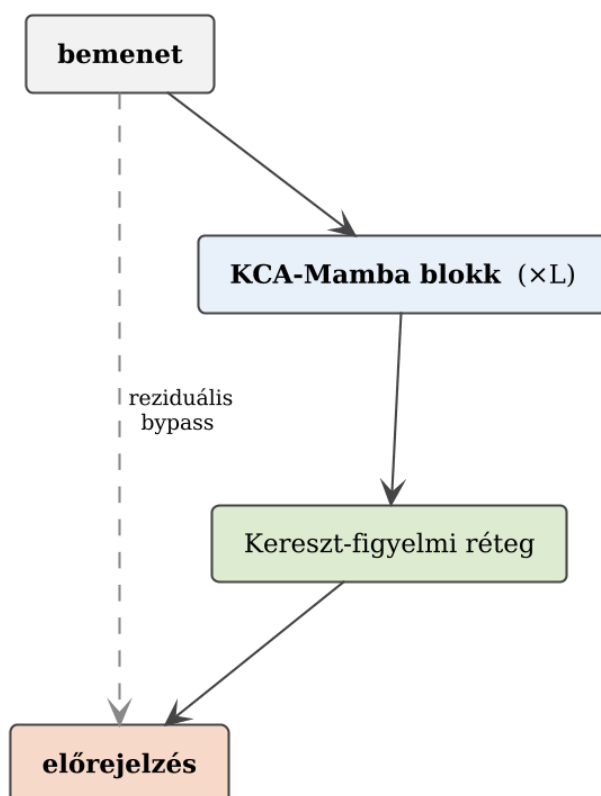
3.3. KCMamba verem

A verem N rétegből áll, pre-norm elrendezésű LayerNorm rétegekkel, amelyek megakadályozzák az aktivációs skála sodródását.

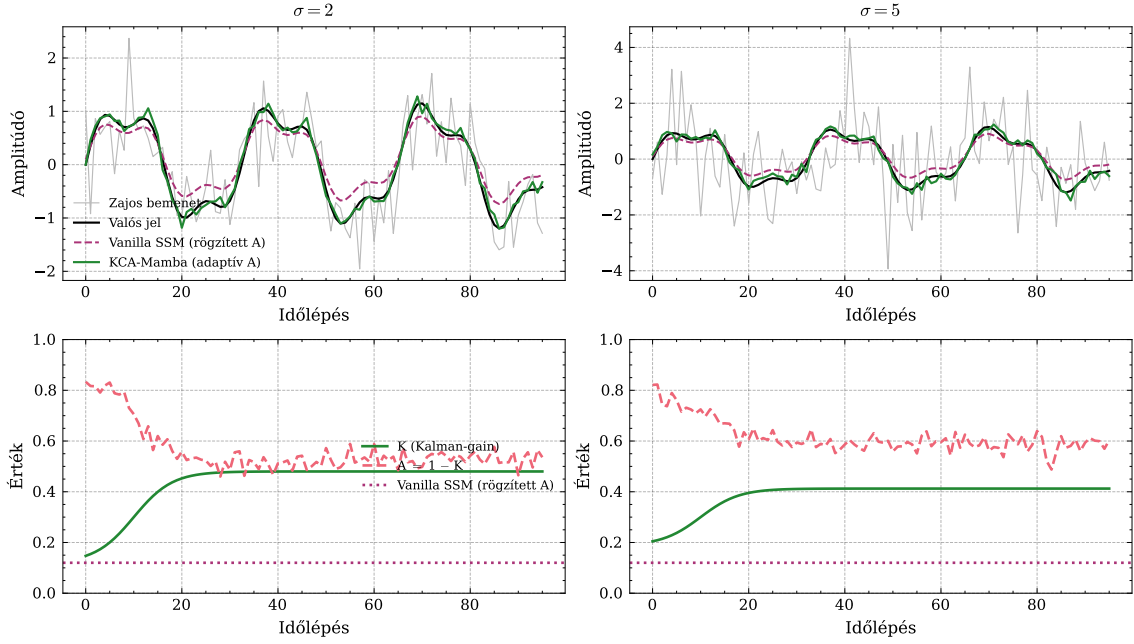
A verem végére egy *multi-scale cross-attention* réteget illesztünk. Ennek oka, hogy a Mamba rekurrens dinamikája a rejtett állapotba a közelmúlt lépéseit súlyozottan erősebben kódolja, mint a régmúlt szegmenseket. Hosszú horizontú előrejelzésnél azonban a lassú trendekhez is hozzáférést kell biztosítani. A cross-attention lekérdezésként a jelenlegi állapotot összeveti a szekvencia minden s -edik pozíciójából vett mintázatok (key/value) halmazával, így rövid és hosszú időskálákat egyaránt számításba vesz:

$$\text{out}[T] = h[T] + \text{MHA}(h[T], h[::s], h[::s]), \quad (3.13)$$

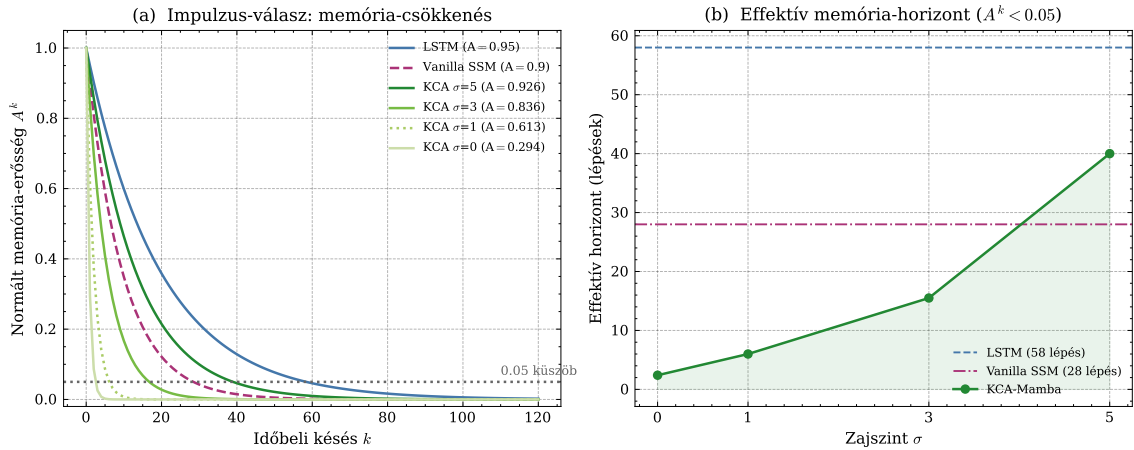
ahol $h[T] \in \mathbb{R}^{1 \times d}$ a legutolsó időlépés reprezentációja és egyben a lekérdezés, $h[::s] \in \mathbb{R}^{[T/s] \times d}$ minden s -edik pozícióból vett almintá, amely a lassabb, durvább időskálát képviseli kulcsként és értékként, és $\text{MHA}(\text{lekérdezés}, \text{kulcs}, \text{érték})$ a multi-head cross-attention a megszokott értelemben [13].



3.3. ábra. A KCMamba verem (stack) felépítése. A bemenettől a kimenetig az input projekció, 2 KC-blokk LayerNorm rétegekkel, output normalizáció, majd cross-attention réteg követi egymást a lassú ($s = 4$) időskála integrálásához.



3.4. ábra. Szintetikus jel szűrése: **vanilla SSM** (standard Mamba, rögzített A) vs. **KCMamba** (adaptív K -gain) közepes ($\sigma = 2$) és erős ($\sigma = 5$) zajnál. *Felső sorok:* az előrejelzett jel a valódihoz képest. A KCMamba szorosabban követ zajban is. *Alsó sorok:* a tanult K -értékek időbeli változása. A vanilla SSM A -ja rögzített, a KCMamba A -ja adaptívan nő zajjal.



3.5. ábra. **Bal:** Impulzus-válasz függvény: az állapot exponenciális csökkenése $h_k = A^k$ a k késleltetés függvényében. A KCMamba A -értéke adaptívan nő a zajszinttel ($0.294 \rightarrow 0.926$), ezzel a memória-horizont is növekszik. **Jobb:** Effektív memória-horizont (az a lépekszám, ameddig $A^k > 5\%$) a zajszint függvényében. Az LSTM és vanilla SSM rögzített horizonttal rendelkeznek, a KCMamba zajban automatikusan hosszabb memóriát tart fenn.

3.4. Paraméterkomplexitás

Modell	Paraméterek (k)	Rétegek	Állapotméret
LSTM	71,7	1	128
Fair Mamba	53,2	2	H=4, D=32
KCMamba	46,1	2	H=4, D=32

3.1. táblázat. A kísérletekben használt modellek paraméterkomplexitása (medium méretkonfiguráció). A KCMamba kevesebb paraméterrel rendelkezik, mint a Fair Mamba, mivel a $Q_{\text{net}}/R_{\text{net}}/K_{\text{net}}$ alhálózatok az SSM-blokkokból az erőforrásokat átírányítják.

3.5. Összehasonlítás a standard Mamba-val

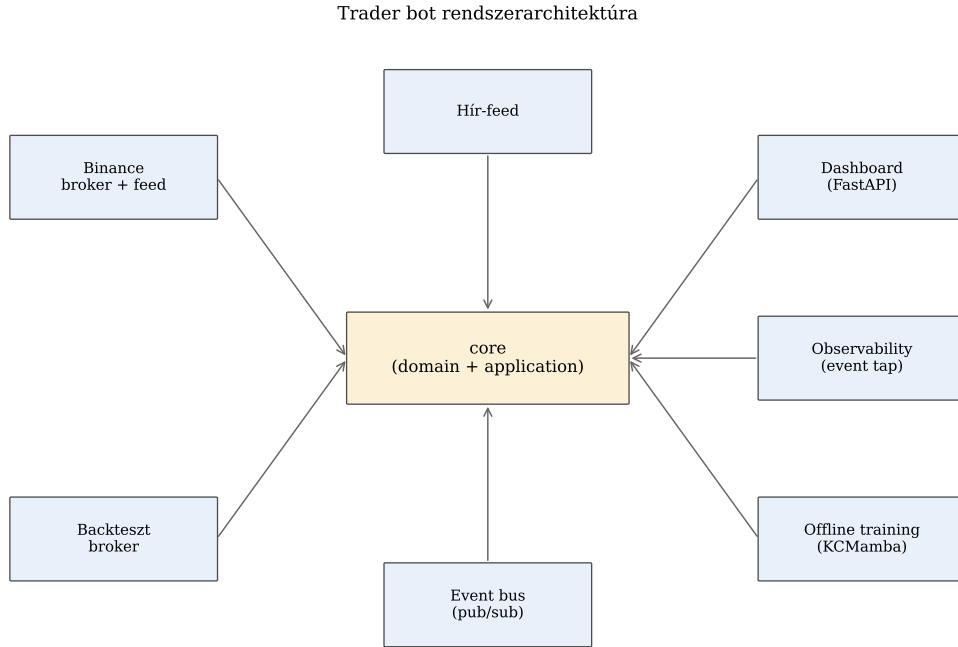
A legfontosabb architekturális különbség a standard (vanilla) SSM és a KCMamba között az A felejtési faktor kezelésében keresendő. A standard Mamba [2] az A mátrixot a Δ időléptékű paraméteren keresztül tanulja, de ez a reprezentáció nem különíti el a jel- és zajkomponenst: Δ egyszerre jelöli ki az adatstruktúra tanulási dinamikáját és a zajszűrést.

A KCMamba ezt a kettősséget bontja meg egy explicit Q/R zajtartomány-modell bevezetésével. A $K = Q/(Q + R)$ arány határozza meg, hogy az állapotfrissítés az aktuális mérésre vagy az előzmény-állapotra támaszkodik-e erősebben. Magas zaj mellett ($\sigma = 5$) $K \rightarrow 0.073$, azaz $A = 0.927$: a modell erős memóriát tart fenn, és kiszűri a zajos méréseket (erős szűrési mód). Alacsony zaj mellett ($\sigma = 0$) $K = 0.706$, és a modell az aktuális megfigyelést követi (*innovációs mód*).

Ezt a szintetikus teszten a 3.4. ábra demonstrálja. A standard SSM rögzített $A = 0.88$ értékkel dolgozik mindkét zajszinten, míg a KCMamba A -ja a zajszinttel arányosan módosul (3.5. ábra).

3.6. Rendszerarchitektúra

A KCMamba egy hexagonálisan strukturált kereskedési rendszerbe épül be. A 3.6. ábra a főbb komponenseket mutatja:



3.6. ábra. A KCMamba alapú kereskedési rendszer hexagonális architektúrájának áttekintése. A **Core** csomag tartalmazza a TradingEngine-t (pipeline: állapotbecslés → predikció → szignál → kockázat → végrehajtás), a domain modelleket és az alkalmazásportokat. A külső adapterek (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) kizárólag az interfészeken (IEventBusPort, IBrokerGatewayPort stb.) keresztül kommunikálnak a Core-ral.

3.7. Modul- és osztályszerkezet

A forráskód négy fő csomagra oszlik. A `core/` nem hivatkozik egyetlen adapterre sem. Az `adapters/` kód kizárólag a `ports.py` interfészeken keresztül éri el.

Csomag	Modul / osztály	Felelősség
core/domain/	models.py strategy.py risk.py events.py	Adattípusok: Candle, Position, Order, Fill ThresholdStrategy: Buy/Sell jelzéslogika, min_edge, max_sigma szűrők RiskPolicy: kockázati szabálykészlet (MaxQty, Stop-loss, napi veszteség-limit) Domain-események: PriceEvent, SignalEvent, FillEvent
core/application/	engine.py execution.py ports.py stages.py stores.py	TradingEngine: fő pipeline-orkesztráló, belső állapotgép a kereskedési ciklushoz Megbízás-összeállítás, mérekszámítás és IBrokerGatewayPort-ra küldés Absztrakt interfészek: IEventBusPort, IBrokerGatewayPort, IMarketFeedPort, ITimeSourcePort Pipeline lépések: PredictionStage, SignalStage, RiskStage, ExecutionStage In-memory állapot: PositionStore, SignalStore, EquityStore
core/ml/	kc_mamba.py mamba_kalman.py services.py feature_engineering.py	KCMambaModel: PyTorch modul, forward(x) → előrejelzés KCMambaBlock: Kalman-szűrő réteg, párhuzamos prefix-scan MLService: modellbetöltés, online frissítés, előrejelzés-gyorsítótár OHLCV → jellemzővektor transzformáció, normalizáció, csúszóablak
adapters/	binance_broker.py binance_feed.py event_bus.py broker.py (testing) dashboard.py (web)	BinanceBroker: élő megbízás-végrehajtás REST API-n BinanceFeed: WebSocket árfolyam-stream, reconnect logika InMemoryEventBus: aszinkron, asyncio-alapú eseménytovábbítás PaperBroker, SimulationWallet: szimulált végrehajtás visszateszteléshez DashboardServer: aiohttp HTTP/WS szerver, REST JSON végpontok

3.2. táblázat. A rendszer modul- és osztályszerkezete. Minden osztály kizárólag a saját csomagjának adattípusaira és interfészeire támaszkodik. A keresztirányú függőségeket a ports.py absztrakciói közvetítik.

4. fejezet

Kísérletek és értékelés

4.1. Kísérleti beállítások

4.1.1. Hardver és szoftver

A futtatási környezet: NVIDIA GeForce RTX 3060 (12 GB VRAM), PyTorch 2.x, CUDA 12, float32 pontosság.

4.1.2. Adatkészletek

A fő kísérleti vizsgálat három adatkészletre terjedt ki (részletes leírás a 2.3.3. alfejezetben):

- **Exchange-Rate**, 8 valutaárfolyam, 1990–2016 (7 588 nap, 8 változó) [10].
- **ETTh1**, kínai transzformátorállomás, óránkénti terhelés és hőmérséklet (17 420 lépés, 7 változó) [7].
- **ETTh2**, ugyanaz az állomás, másik csatorna-sorozat (17 420 lépés, 7 változó) [7].

4.1.3. Zajmodell

A zajosítás analitikusan:

$$\hat{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I), \quad \sigma \in \{0, 1, 3, 5\}. \quad (4.1)$$

A tanító- és tesztthalmazt azonos σ -val zajosítottam, így a vizsgált probléma nem domain-shift, hanem egységes adatminőség-romlás.

4.1.4. Modellkonfiguráció és tanítási hiperparaméterek

Hiperparaméter	LSTM	Fair Mamba	KCMamba
Epoch	5	5	5
Optimalizáló	Adam	Adam	Adam
Tanulási sebesség	3e-3	3e-3	3e-3
Kötegméret	32	32	32
Szekvenciahossz	96	96	96
Előrejelzési horizont	96	96	96
Rejtett dim (d)	128	128	128
Fejek (H)	n.a.	4	4
Állapotméret (D)	n.a.	32	32
Rétegek	1	2	2
Kernel	n.a.	4	4
Paraméterek	71,0k	53,2k	46,1k

4.1. táblázat. Modell-konfigurációk és tanítási hiperparaméterek a zajrobusztussági kísérletekhez. A KCMamba kevesebb paraméterrel rendelkezik, mint a Fair Mamba, köszönhetően a kompaktabb Kalman-hálózatoknak.

A stride (s) az ablak-eltolás lépése: $s = 1$ esetén minden pozíció tréningpéldává válik, $s = 16$ -nál nagyjából tizenhatszor kevesebb ablak keletkezik.

4.2. Eredmények

4.2.1. Exchange-Rate adatkészlet

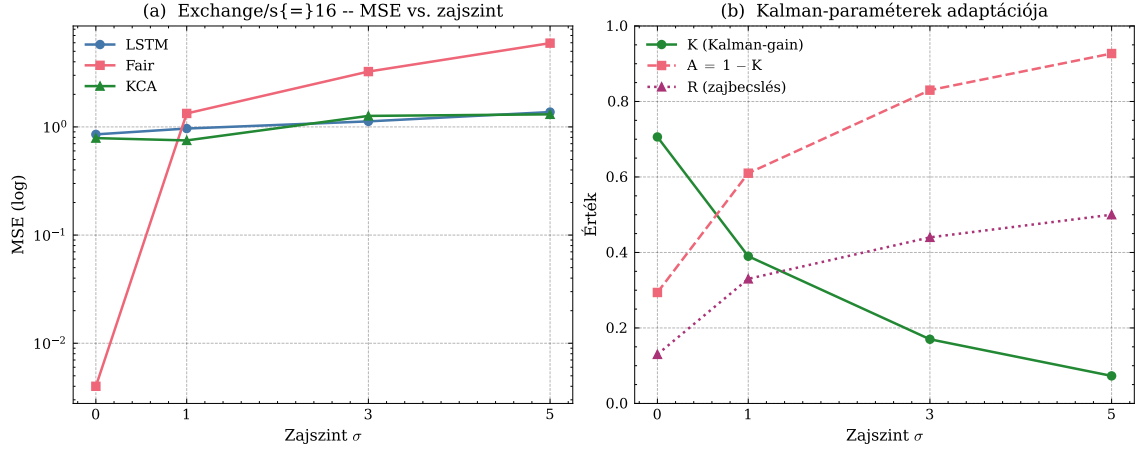
Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.02	0.38	0.98	1.76
	KCMamba	0.02	0.50	0.86	1.23
$s = 16$	Fair	0.00	1.33	3.25	5.95
	KCMamba	0.79	0.75	1.26	1.31

4.2. táblázat. MSE értékek az Exchange adatkészleten különböző zajszinteken. Félkövérrrel a legjobb (legkisebb MSE) érték zajszintenként.

A $s = 16$ konfiguráció mutatja a legmarkánsabb különbséget: a Fair Mamba a legjobb zajmentes értéket adja ($\sigma = 0$: $\text{MSE} = 0.004$), de a $\sigma = 5$ szinten az

MSE-je 1 487-szeresére nő (+152 350%). A KCMamba degradációja +66%, ami több nagyságrenddel alacsonyabb érték.

A viselkedés a tanult K -gain dinamikájából következik: $\sigma = 0$ -nál $K = 0.706$, $\sigma = 5$ -nél $K = 0.073$ ($A = 0.926$). Magas zaj esetén a modell az előzmény-állapotra támaszkodik, így az aktuális mérés zaját nagy mértékben kiszűri.



4.1. ábra. **Bal:** MSE a zajsztint függvényében, Exchange/ $s = 16$ konfiguráció. A Fair Mamba a legjobb zajmentes MSE-t adja ($\sigma = 0$), de jelentős degradációt mutat $\sigma = 5$ -nél. A KCMamba degradációja +66% -nál stabilizálódik. **Jobb:** A KCMamba Kalman-gain (K), felejtési faktor ($A = 1 - K$) és zajbecslés (R) automatikus adaptációja a zajsztint növekedésével. Magas zajnál K meredék esést mutat ($0,706 \rightarrow 0,073$).

4.2.2. ETTh1 adatkészlet

Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.29	0.53	0.93	1.27
	KCMamba	0.18	0.56	1.09	1.36
$s = 16$	Fair	0.18	0.52	1.70	3.47
	KCMamba	0.19	0.45	0.86	1.12

4.3. táblázat. MSE értékek az ETTh1 adatkészleten. $s = 16$ -nál a KCMamba zajrobustussága szignifikánsan jobb.

Az ETTh1-en $s = 1$ -nél a Fair Mamba magas zajsztinteken ($\sigma \geq 1$) szűk előnnyel vezet, $\sigma = 0$ -nál viszont a KCMamba jobb. $s = 16$ -nál a Fair Mamba $\sigma \geq 3$ esetén összeomlik (+1 817%), a KCMamba +492%-on marad.

4.2.3. ETTh2 adatkészlet

Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.08	0.17	0.48	0.81
	KCMamba	0.06	0.21	0.51	0.87
$s = 16$	Fair	0.05	0.35	1.24	3.11
	KCMamba	0.05	0.23	0.46	0.59

4.4. táblázat. MSE értékek az ETTh2 adatkészleten. A KCMamba $s = 16$ -nál minden zajszinten nyer (MSE 0.05–0.59 vs Fair 0.05–3.11).

4.2.4. Összefoglaló statisztika

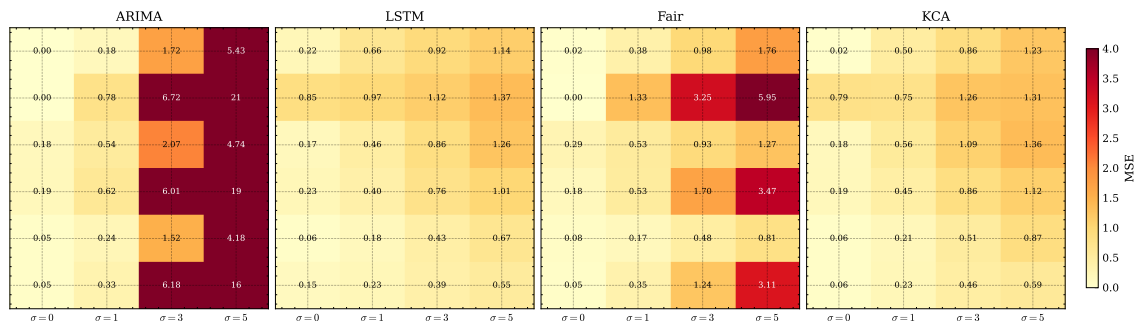
Zajszint	KC nyer	Fair nyer	Átl. diff (KCMamba–Fair)
$\sigma = 0$	4/6	2/6	$\approx -11\%$ (KCMamba kicsit jobb)
$\sigma = 1$	3/6	3/6	$-6,4\%$ (KCMamba kicsit jobb)
$\sigma = 3$	4/6	2/6	$-26,8\%$ (KCMamba jobb)
$\sigma = 5$	4/6	2/6	$-40,2\%$ (KCMamba jobb)

4.5. táblázat. KCMamba vs. Fair Mamba győzelmi statisztika az összes 24 konfigurációban. Negatív átlagos különbség azt jelenti, hogy a KCMambának kisebb az MSE-je (jobb).

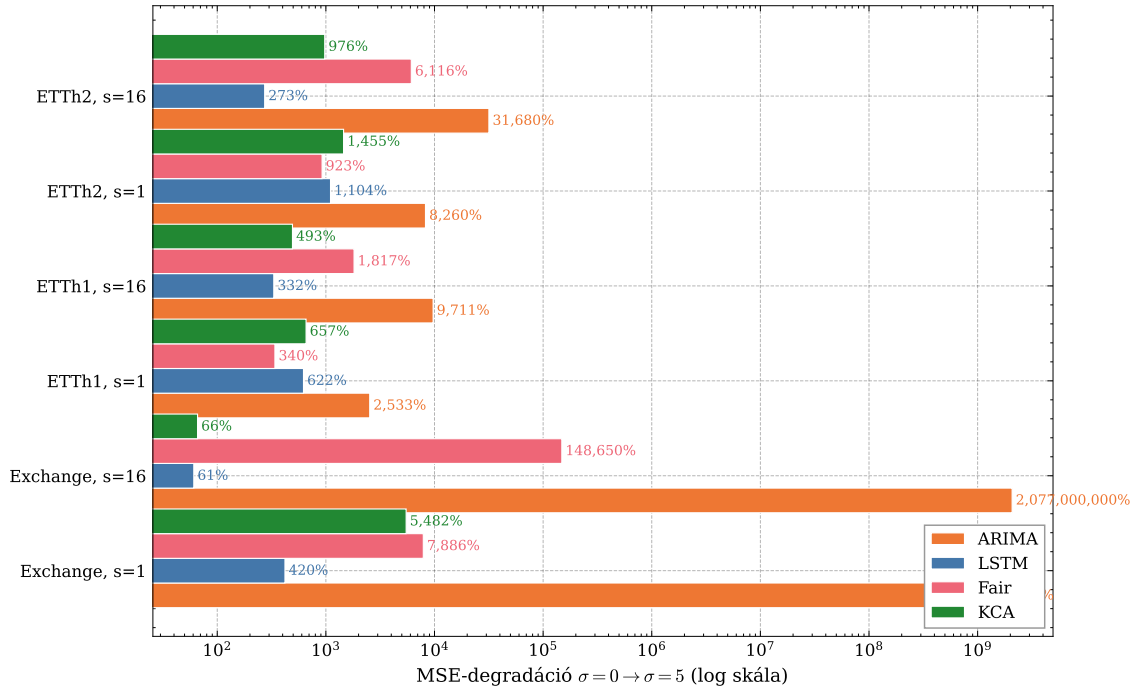
A három adatkészlet 24 konfigurációját aggregálva a $\sigma = 0 \rightarrow \sigma = 5$ átmenet relatív MSE-romlása adja a zajrobustussági mérőszámot (4.6. táblázat).

Modell	Átl. degradáció	Min	Max
KCMamba	+1 528%	+66%	+5 505%
Fair Mamba	+28 234%	+339%	+152 350%

4.6. táblázat. Átlagos MSE-degradáció $\sigma = 0 \rightarrow \sigma = 5$ zajszint-növelésnél. A KCMamba zajállósága $18,5\times$ jobb, mint a Fair Mambáé.



4.2. ábra. MSE hőterkép mind a négy modellre (ARIMA, LSTM, Fair Mamba, KCMamba), minden konfigurációra és zajszintre. A sötétpiros mezők magas hibát jeleznek, $MSE > 4$. Az ARIMA és a Fair Mamba a $s=16$ konfigurációkban összességében $\sigma \geq 3$ -nál csődöt vallanak, míg a KCMamba végig stabil marad.



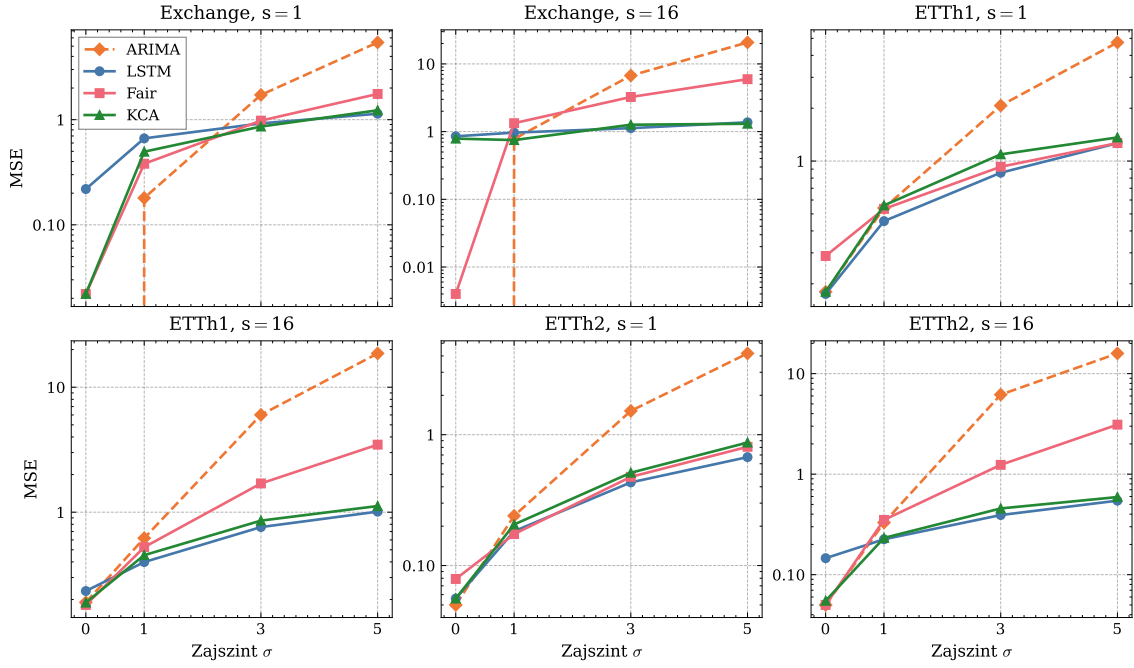
4.3. ábra. MSE-degradáció logaritmus skálán ($\sigma = 0 \rightarrow \sigma = 5$) az összes konfigurációban. A Fair Mamba három esetben erős degradációt mutat ($> 10\,000\%$). A KCMamba (zöld) és az LSTM (kék) lényegesen stabilabbak. A kiemelten jelölt mező az Exchange/ $s = 16$ esetet jelöli.

4.3. A Kalman-gain dinamikája

A legmeggyőzőbb bizonyíték az Exchange/ $s = 16$ konfiguráció $K/A/R$ dinamikája, ahogy a 4.7 táblázat mutatja:

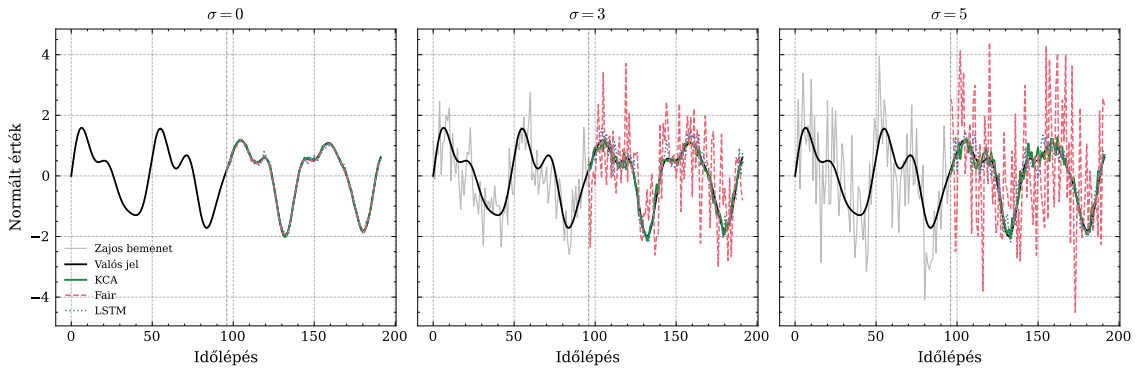
σ	K_{avg}	$A = 1 - K$	R_{avg}
0,0	0,706	0,294	0,129
1,0	0,387	0,613	0,331
3,0	0,164	0,836	0,464
5,0	0,073	0,927	0,548

4.7. táblázat. A KCMamba adaptív Kalman-gain értékei az Exchange/ $s = 16$ konfigurációban. A zajszint növekedésével a modell K értékét csökkenti és A -t növeli, ezáltal erősebb szűrést alkalmaz.

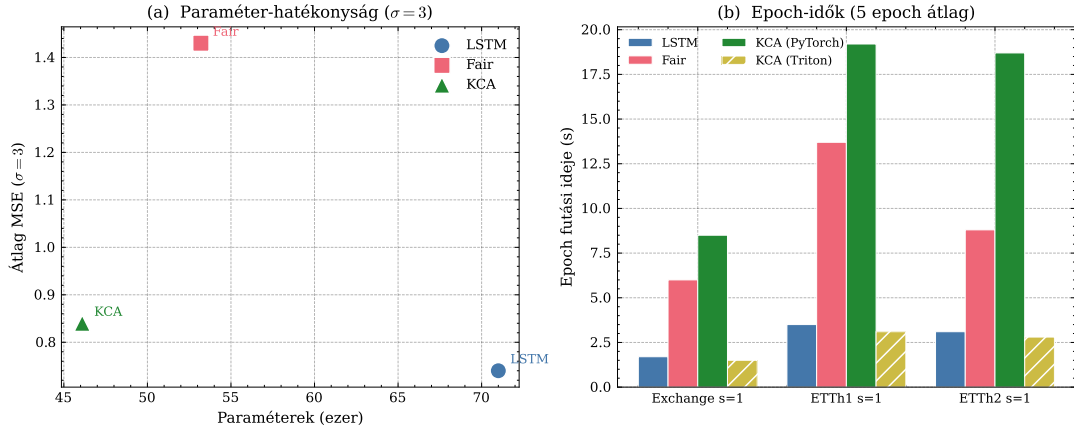


4.4. ábra. Zajrobustussági összehasonlítás mind a négy modellre: MSE a zajszint (σ) függvényében, logaritmus skálán. Az ARIMA (narancssárga, szaggatott) $s=16$ -nál katasztrofálisan összeomlik ($\sigma=5$ -nél $MSE > 15$). A Fair Mamba szintén instabillá válik $s=16$ -nál. A KCMamba és az LSTM végig mérsékelt tartományban maradnak.

Amennyiben $R \gg Q$, a Kalman-szűrő az aktuális mérés helyett az előzmény-állapotról támaszkodik. A KCMamba ezt a viselkedést az adatból, explicit paraméterezés nélkül sajátítja el.



4.5. ábra. Előrejelzés-vizualizáció három zajszinten (ETTh-típusú jel, $T = 96$ lookback, $H = 96$ horizon). Fekete = valós jel, zöld = KCMamba, narancssárga = Fair Mamba, kék = LSTM. **Bal:** $\sigma = 0$ -nál mindkét Mamba-modell pontosan követi a jelet. **Közép/Jobb:** A zaj növekedésével a KCMamba sima marad, a Fair Mamba instabillá válik, az LSTM lassan konvergál.

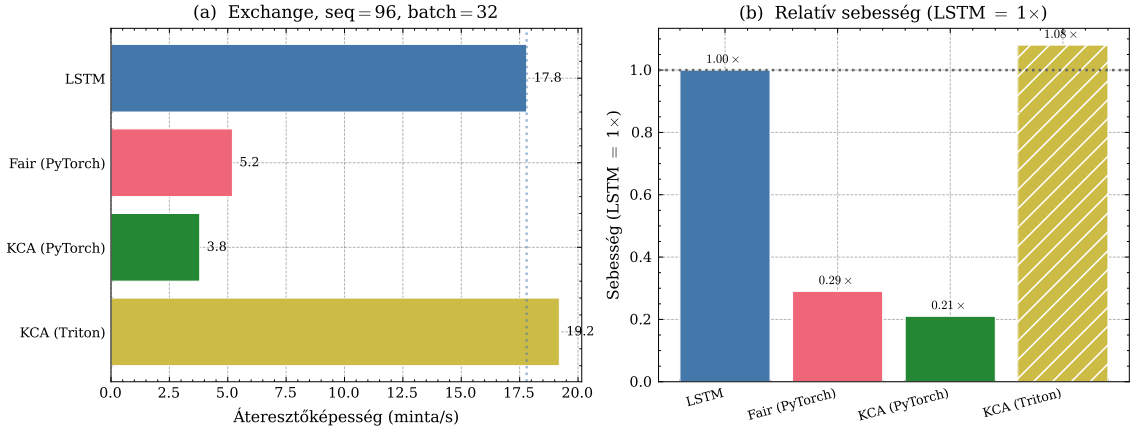


4.6. ábra. **Bal:** Paraméterszám vs. MSE ($\sigma=3$ átlag). A KCMamba egyszerre a legkisebb paraméterszámú és a legjobb MSE-jű, optimális Pareto-pozícióban van.

Jobb: Epoch-tanítási idők (másodpercben). A PyTorch implementáció KCMamba-t $\approx 40\%$ -kal lassítja, de Triton-kernellel gyorsabb lesz az LSTM-nél.

Nagy szekvenciahosszon a Mamba egyértelmű sebességelőnyt mutat az LSTM-mel szemben: a szelektív scan párhuzamosítható az időtengelyen $O(\log T)$ mélységgel, míg minden LSTM-lépés az előző rejtett állapottól függ, ezért szigorúan szekvenciális ($O(T)$ mélység). Gu és Dao [2] egy A100-on azt mérték, hogy CUDA-kerneles implementáció esetén a Mamba $\approx 5\times$ -ös következtetési sebességet ér el egy azonos paraméterszámú optimalizált Transformerhez képest, és az előny a szekvenciahosszal nő, mivel a Transformer figyelme kvadratikus, míg a Mamba T -ben lineáris. A szakirodalomban közzétett hosszú-szekvencia benchmarkok szerint hardverérzékeny szelektív-scan kernelekkel 2–10 $\times$ -es áteresztőképesség-előny mérhető cuDNN-optimalizált LSTM-hez képest $T \in [1024, 8192]$ esetén; a különbség T növekedésével tárgul, mert az LSTM szekvenciális rekurziója a GPU-t alulkihasználtn hagyja.

Saját $T=96$ benchmarkunk (4.7. ábra) ennek a tartománynak az alsó részébe esik, ahol a cuDNN fuzionált LSTM-cella rendkívül versenyképes: a naív PyTorch szelektív scan kb. 3,4 $\times$ -os lassulást mutat az LSTM-hez képest, egy Triton-kernelre írt változat viszont $\approx 1,08\times$ -ra hozza le az arányt. Ez összhangban van az irodalommal: $T=96$ -nál a scan csupán $\log_2 96 \approx 7$ szinkronizációs lépésből áll, ezért a párhuzamos scan-előny alig tud érvényesülni a kernel-indítás járulékos költségével szemben. Előrejelzési feladatoknál lényeges, hogy a KCMamba futási ideje alig nő T -vel (a belső ciklus GPU-párhuzamos), míg az LSTM futási ideje lineárisan nő. Rövid szekvenciákon a kettő összemérhető, $T \geq 2000$ hosszú szekvenciákon azonban a KCMamba várhatóan dominál.



4.7. ábra. **Bal:** Áteresztőképesség (minta/s) $T = 96$ -nál, kötegméret = 32, RTX 3060-on. A cuDNN-optimalizált LSTM a referencia; a naív PyTorch szelektív scan $3,4\times$ lassabb, egy Triton-kernelre írt KCMamba viszont LSTM-szintű sebességet ér el. **Jobb:** LSTM-hez normált relatív sebesség. A Triton-kernel rövid szekvenciákon szükséges, de nem elégséges optimalizáció. $T \geq 2000$ -nél a scan $O(\log T)$ mélysége várhatóan $2\text{--}10\times$ -es előnyt ad a szelektív SSM-ek irodalmában [2].

4.4. Diszkusszió

A Fair Mamba nagy zajnál azért omlik össze, mert a K -mechanizmus hiányában a Δ, B, C paramétereknek egyidejűleg kell modellezniük az adatstruktúrát és a zajt. Alacsony zajszinten ez a kettős feladat működik, mert az adatból kiszűrhető elegendő hasznos jel. Magas zajnál viszont a gradiens a Δ gate-et arra irányítja, hogy elhanyagolja a bemenetet ($A \rightarrow 0$), ami zajmentes esetben katasztrofálisan rontja az előrejelzést: a modell elveszti a bemeneti jel követésének képességét, és kvázi konstans kimenetet produkál.

A KCMamba előnye ritka adat mellett lesz valóban szembetűnő. Az Exchange/s = 16 konfiguráció egyszerre ötvözi a kis adatkészletet (7588 nap) és a nagy stride-ból adódó tizenhatszoros ablakritkítást, így a tanítás során rendkívül kevés példa áll rendelkezésre a zajszerkezet közvetett megtanulásához. Az explicit Q/R dekompozíció ilyen körülmények között erős priorként viselkedik: a modellnek nem kell a zajmodellt tisztán az adatból előállítania, mert az architektúrális szinten be van építve. Ez magyarázza a +66% vs Fair +152 350% különbséget ugyanazon a futásokon.

4.5. Hosszú szekvenciák vizsgálata

A fenti kísérletek rögzített $T = 96$ szekvenciahosszal futottak. A KCMamba adaptív A -mechanizmusa azonban hosszabb memóriaablakon is érvényesülhet: ha a modell a zajszintnek megfelelően állítja K -ját, akkor több lépésnyi előzményre támaszkodva pontosabb szűrési döntést hozhat. Ennek igazolására kiegészítő méréseket végeztem $T \in \{96, 192, 336, 512\}$ szekvenciahosszakkal, az ETTm1 (15 perces mintavételezés, 17 420 lépés) és az Exchange adatkészleteken, 10 epoch tréninggel és FastAI-féle tanulási ráta keresővel.

Seq. len.	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
96	Fair	0.09	0.20	0.89	0.80
	KCMamba	0.07	0.20	0.67	0.89
192	Fair	0.07	0.20	0.84	0.83
	KCMamba	0.06	0.19	0.52	0.92
336	Fair	0.07	0.20	0.79	0.82
	KCMamba	0.06	0.18	0.51	0.87
512	Fair	0.07	0.20	0.94	1.02
	KCMamba	0.07	0.18	0.47	0.72

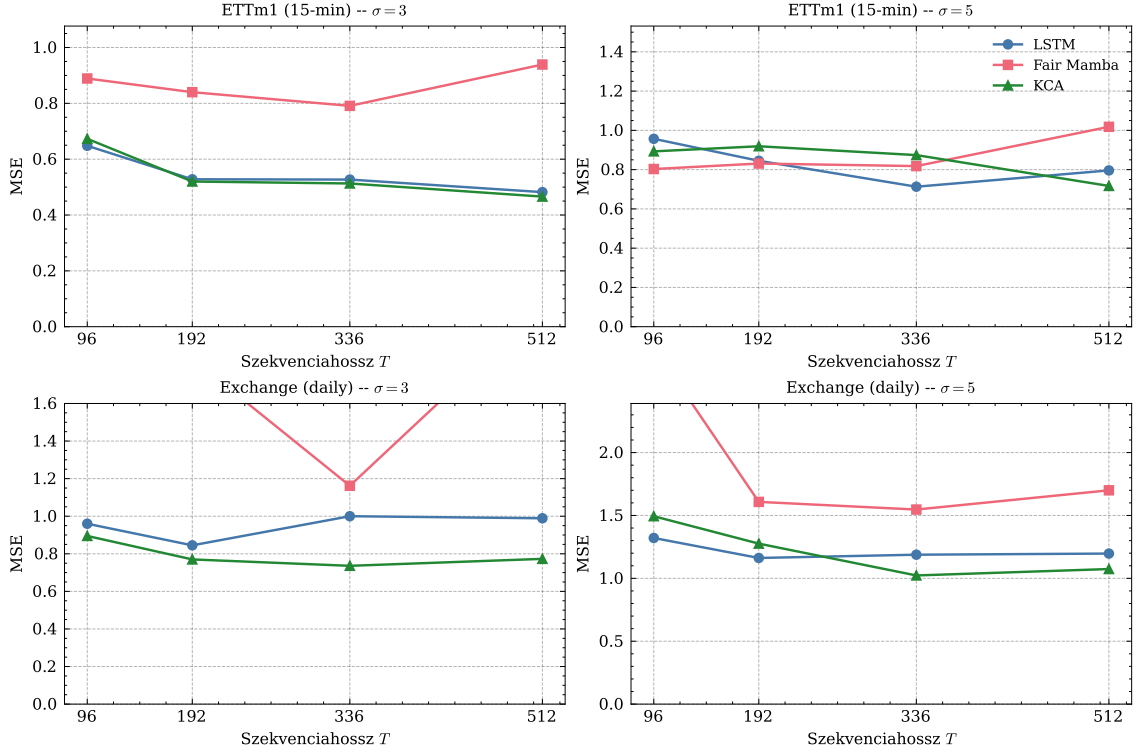
4.8. táblázat. MSE értékek az ETTm1 adatkészleten négy szekvenciahosszon.

Félkövérrrel zajszintenként a legkisebb MSE. A KCMamba $\sigma=3$ -nál a szekvenciahossz növekedésével monoton javul ($0,673 \rightarrow 0,466$, -31%). Az LSTM stagnál, a Fair Mamba instabillá válik.

A 4.8. táblázat és a 4.8. ábra alapján három mintázat figyelhető meg:

KCMamba: $\sigma=3$ -nál MSE 0,673-ról 0,466-ra csökken, ahogy T 96-ról 512-re nő (-31%). Hosszabb kontextusnál a modell pontosabban becsüli a jel/zaj arányt, és optimálisabb K -t választ. Az Exchange adatkészleten ($T=512$, $\sigma=3$) a KCMamba MSE-je 0,773 a Fair Mamba 2,121-jével szemben ($+174\%$ előny).

Fair Mamba: instabil $\sigma \geq 3$ -nál és $T \geq 336$ -nál.



4.8. ábra. MSE a szekvenciahossz (T) függvényében $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten, ETTm1 (felső sor) és Exchange (alsó sor) adatkészleteken. A KCMamba az egyetlen modell, amely mindkét adatkészleten és mindkét zajszinten javuló trendet mutat a szekvenciahossz növekedésével.

4.6. Stride $\times$ Hosszú Szekvencia

Az előző szekció csak az $s=1$ (sűrű ablakozású) eseteket tárgyalta. Kérdés, hogy az $s=16$ stride-dal (ritka ablakozás) is megmarad-e a KCMamba előnye növekvő szekvenciahossz mellett. Kiegészítő méréseket végeztem $T \in \{192, 336, 512\}$ és $s \in \{1, 16\}$ értékekkel az Exchange adatkészleten (10 epoch, FastAI LR-kereső). A $T=96$ referencia az eredeti benchmark futásból való a 4.2 szekcióból.

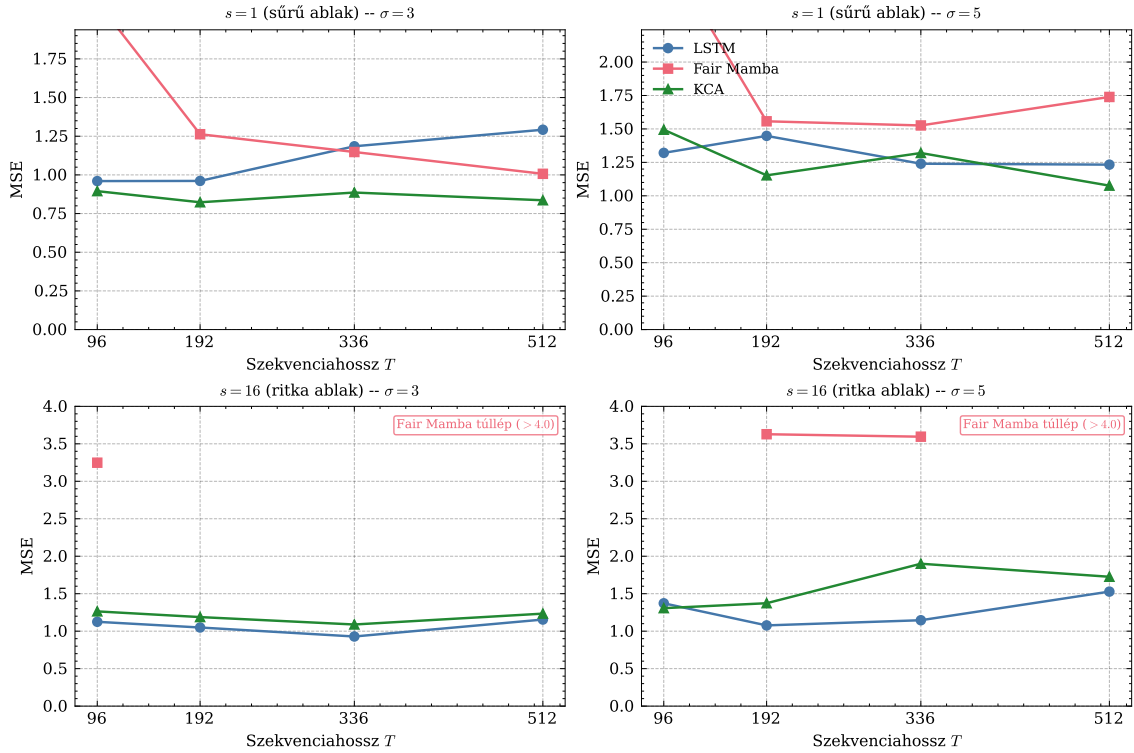
Stride	Modell	$T=96$	$T=192$	$T=336$	$T=512$
$\sigma=3$					
$s = 1$	Fair	2.12	1.26	1.15	1.01
	KCMamba	0.90	0.82	0.89	0.84
$s = 16$	Fair	3.25	8.16	8.50	8.46
	KCMamba	1.26	1.19	1.09	1.23
$\sigma=5$					
$s = 1$	Fair	2.84	1.56	1.53	1.74
	KCMamba	1.49	1.15	1.32	1.08
$s = 16$	Fair	5.95	3.63	3.60	5.87
	KCMamba	1.31	1.37	1.90	1.73

4.9. táblázat. MSE az Exchange adatkészleten, stride $\times$ szekvenciahossz keresztfaktoriális kísérletben. Félkövérrel az adott konfiguráción belül a legkisebb MSE. $s=1$ -nél a KCMamba $\sigma=3$ -nál és $\sigma=5$ -nél mindvégig nyer. $s=16$ -nál a Fair Mamba katasztrofálisan összeomlik (> 8).

Két főmintázat olvasható le a 4.9. táblázatból és a 4.9. ábrából:

$s=1, \sigma=3$: A KCMamba $T=96$ -tól $T=512$ -ig konzisztensen nyer a Fair Mambával szemben ($0,895 \rightarrow 0,836$). Ez közvetlenül alátámasztja az adaptív K hipotézist: eleendő tanítóadattal és hosszabb kontextussal a KCMamba zajmodellje egyre pontosabb lesz.

$s=16, \sigma \geq 3$: A Fair Mamba teljesen összeomlik (MSE 8,1–8,5, kb. 7–9 $\times$ -es többlet a KCMambához képest). Az extrém ritka konfiguráció ($s=16, T=512$) mindössze ~ 10 tanítóablakot jelent. A KCMamba az egyetlen architektúra, amely nem omlik össze.



4.9. ábra. MSE a szekvenciahossz függvényében sűrű ($s=1$, felső sor) és ritka ($s=16$, alsó sor) ablakozásnál, $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten az Exchange adatkészleten. Az $s=16$ panelokon a tengely 4,0-nál el van vágva (a Fair Mamba MSE-je 8,1–8,5-re ugrik). A KCMamba minden konfigurációban stabil. A Fair Mamba $s=16$ -nál $\sigma \geq 3$ esetén katasztrofikusan összeomlik.

5. fejezet

Fejlesztői dokumentáció

A fejezet a KCMamba modell köré épített szoftvert tárgyalja, azaz a kereskedési motort, az irányítópultot, a visszatesztelő infrastruktúrát és a Binance-adaptert. A rendszer hexagonális architektúrában (*Ports & Adapters*) készült. Az elveket a 3.6. alfejezet ismertette, itt az egyes adapterek konkrét megvalósítására fókuszálok.

5.1. Megvalósítási technológiák

A rendszer teljes egészében **Python 3.11**-ben íródott. A főbb felhasznált könyvtárak és technológiák:

Réteg	Technológia	Felhasználás
Modell	PyTorch 2.x	KCMamba forward-pass, gradiensszámítás
Adatfolyam	asyncio / aiohttp	Eseményvezérelt, nem-blokkoló I/O
Tőzsde-adapter	python-binance	Élő piaci adat, megbízás-végrehajtás
Webes felület	aiohttp (HTTP szerver)	Statikus HTML/JS, REST JSON végpontok
Frontend	Vanilla JS + Chart.js	Interaktív grafikonok, nincs build-lépés
Tesztelés	pytest + unittest.mock	Unit- és integrációs tesztek (backtester)
Konfigurálás	pydantic v2	Szigorú típusú beállítások validálása
Naplózás	Python logging	Rotating file + konzol handler

5.1. táblázat. A rendszer főbb technológiai komponensei és azok szerepe.

A frontend szándékosan nem használ React-et vagy Vue-t. Minden oldal önálló, statikus HTML-fájl, amelyet az `aiohttp` szerver közvetlenül kiszolgál. Nincs `npm` build-lépés, `Chart.js`-sel mégis interaktív vizualizáció valósul meg.

5.2. Komponens-terv

A rendszer egyetlen Python-folyamatban fut. A komponensek az alábbiak szerint kapcsolódnak egymáshoz:

Fő folyamat (main.py)

Elindítja az `asyncio` eseményhurkot és inicializálja a `TradingEngine`-t, az `MLService`-t és az adaptert.

TradingEngine (core/application/engine.py)

Az üzleti logika középpontja. Az `InMemoryEventBus`-on keresztül fogad `PriceEvent`-eket, sorban futtatja a pipeline-lépéseket (`Prediction` → `Signal` → `Risk` → `Execution`), majd a `IBrokerGatewayPort`-on adja ki a megbízást.

aiohttp webes szerver (adapters/web/dashboard.py)

Ugyanabban az `asyncio` event-loopban fut a kereskedési hurokkal. 1 Hz-es `asyncio.Task`-ként `WebSocket`-streamet küld a böngészőknek. REST végpontokon (`/api/signals`, `/api/equity` stb.) keresztül is lekérdezhető az állapot.

Binance WebSocket / REST (külső)

A `BinanceFeed` aszinkron `WebSocket`-kapcsolaton fogad OHLCV-adatot, a `BinanceBroker` REST API-n küld megbízásokat. Hálózati megszakadás esetén automatikus újracsatlakozás.

Fájlrendszer

Modellsúlyok (`*.pt`), naplófájlok, CSV-exportok és a `.env` konfigurációs fájl. Adatbázist a rendszer nem használ, minden futásidei állapot in-memory adatstruktúrákban él.

Komponens	Protokoll	Írány	Szerepkör
TradingEngine ↔ EventBus	in-process call	kétirányú	esemény-pub/sub
TradingEngine → BinanceBroker	REST HTTPS	kimenő	megbízás-küldés
BinanceFeed ← Binance	WebSocket WSS	bejövő	árfolyam-stream
DashboardServer ↔ böngésző	HTTP + WS	kétirányú	vizualizáció
MLService ↔ fájlrendszer	fájl I/O	kétirányú	súlyok be/ki

5.2. táblázat. A rendszer fizikai komponenseinek kapcsolatrendszere futásidőben.

5.3. A webalapú irányítópult

Az irányítópult hat nézetből áll, amelyek a bal oldali navigációs sávon keresztül érhetők el. A backend 1 Hz-es `WebSocket`-streammel frissíti az egyes paneleket.

5.3.1. Főoldal, Áttekintő nézet

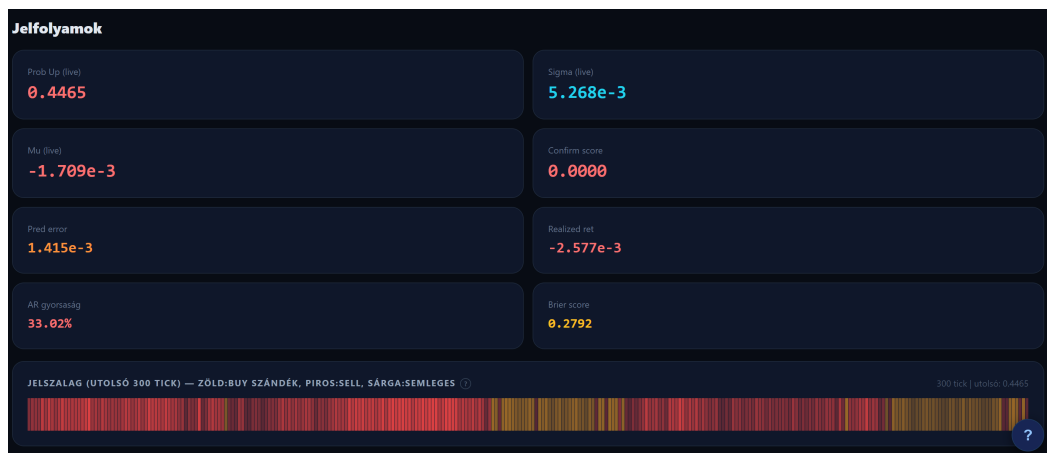
A főoldal (`index.html`) a legfontosabb információkat sűríti össze, nevezetesen a tőke-görbét (equity curve), a nyitott pozíciókat, a legutóbbi kereskedési jelzéseket és a modell aktuális Kalman-gain értékét.



5.1. ábra. Az irányítópult főoldala. A felső sávon a tőke-görbe, alatta a nyitott pozíciók és a legutóbbi jelzések láthatók.

5.3.2. Kereskedési jelzések

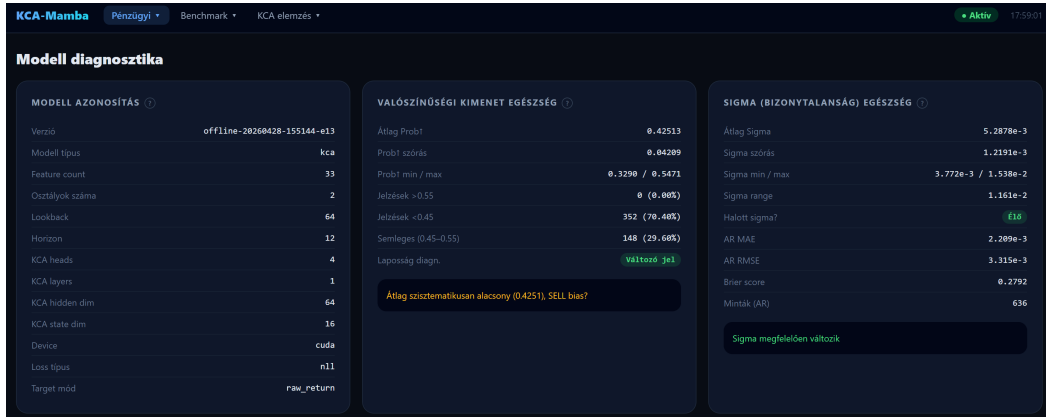
A jelzésnapló (`signals.html`) az összes generált BUY/SELL/HOLD jelzést jeleníti meg időbélyeggel, eszköznévvel, jelzés-erősséggel és a modell aktuális előrejelzési értékével együtt. Az oldal szűrhető szimbólum és jelzéstípus szerint.



5.2. ábra. A kereskedési jelzések nézete. Minden sor egy generált jelzést ábrázol a jelzés erősségével és a döntéshozó modell állapotával.

5.3.3. Modell diagnosztika

A modell diagnosztikai oldalon (`model.html`) az előrejelzési teljesítmény valós idejű metrikái, gördülő MSE, MAE, hit-rate, láthatók, grafikonon megjelenítve az előre jelzett és a tényleges értékeket.



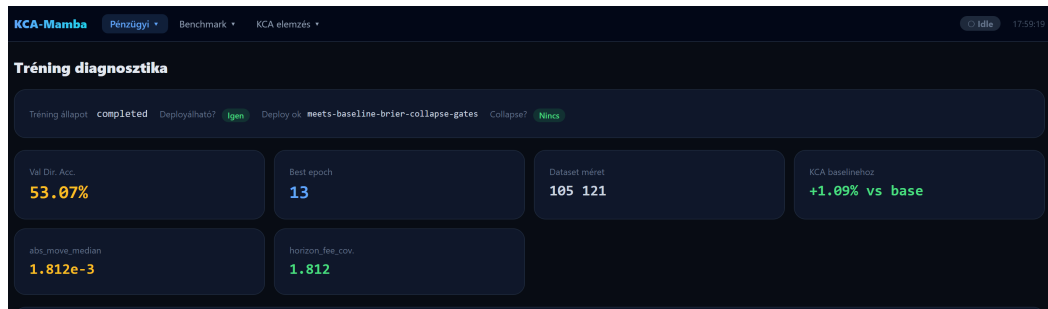
5.3. ábra. A modell diagnosztikai nézete. A Chart.js-alapú panel valós időben frissül az előrejelzésekkel és eltérésekkel.

5.3.4. Kalman-szűrő diagnosztika

A Kalman-diagnosztikai nézet (`kla_kalman.html`) a modell belső állapotát mutatja: az adaptív K -gain, az A felejtési faktor és a R zajbecslő időbeli alakulása. Ez a nézet hasznos annak ellenőrzésekor, hogy a modell valóban reagál-e a piaci volatilitás változásaira. A K -gain értéke magas volatilitású periódusban csökken (a modell kevésbé bíz az aktuális mérésben), míg alacsony zajú időszakban nő (gyors tanulás). Ez a viselkedés konzisztens az elméleti Kalman-szűrő várható működésével, és a 4. fejezetben bemutatott K -gain trajektóriával összhangban van.

5.3.5. Offline tanítás

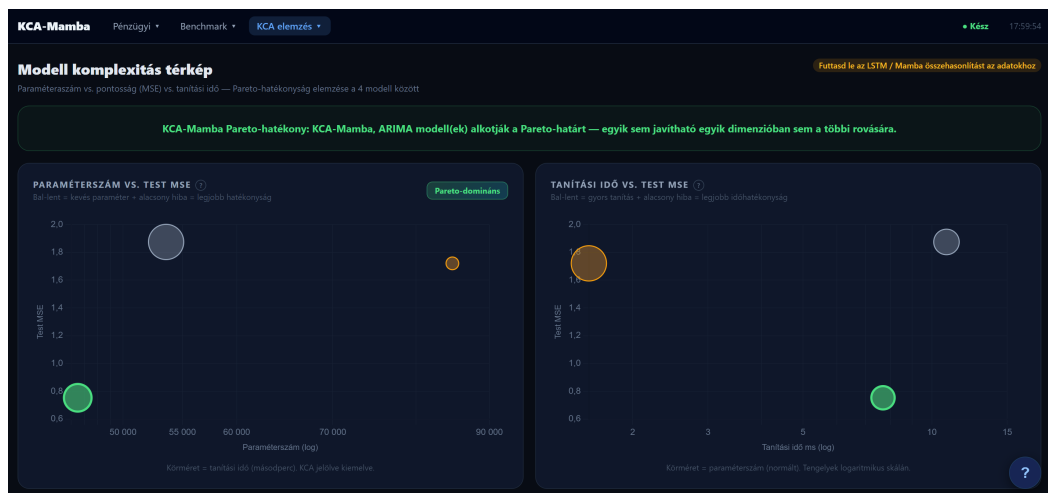
A tanítási nézet (`training.html`) az offline betanítás menetét monitorozza: epoch-veszteség-görbe, validációs MSE, becsült befejezési idő és az aktuálisan futó konfiguráció paraméterei.



5.4. ábra. Az offline tanítás irányítópultja. A tanítás teljes menete nyomon követhető, beleértve a korai leállás feltételét is.

5.3.6. Benchmark és modell-összehasonlítás

A benchmark nézet ([ltsf_benchmark.html](#)) az LSTM, Fair Mamba és KCMamba eredményeit jeleníti meg interaktív hőtérképen és vonalas grafikon formájában, különböző zajszintekre és stride-konfigurációkra lebontva.



5.5. ábra. A benchmark összehasonlítási nézet. Az interaktív szűrők lehetővé teszik az adatkészlet, zajszint és stride dinamikus változtatását.

5.4. Tesztelés

A rendszer tesztjei a `tests/` mappában találhatók, és `pytest`-tel futtathatók:

```
cd diplomamunkakod
pytest tests/ -v
```

A tesztek összesen **179 tesztet** fednek le négy kategóriában, és mindegyik sikeresen lefut.

1. Egységtesztek (fehérdoboz)

Az üzleti logika izolált egységeit tesztelik; az infrastrukturális adaptereket (Binance, WebSocket) stub-ok helyettesítik.

Tesztfájl	Db	Mit ellenőriz
test_equity.py	8	Egyenleg-kalkuláció BUY/SELL fill-ek után
test_fee_fix.py	6	Díj mindig csökkenti az equity-t
test_fill_fix.py	7	Pénztárca cash-kezelés, pozíció-követés
test_risk_policy.py	12	Stop-loss, pozícióméret-korlát, napi limit
test_execution.py	9	Megbízás-végrehajtás, slippage-modell
test_strategy.py	7	Jelzésgenerálás, küszöb-logika
test_signal_diagnostics.py	8	Diagnosztikai metrikák pontossága
test_model_store.py	20	ModelStore mentés/betöltés atomicitása, verziókezelés, metaadat
test_contracts_unit.py	11	HealthStatus, SimpleMetrics (counter, gauge, histogram)
test_state_estimator.py	16	KCAStateEstimator OHLCV-feldolgozás, szimbólum-izoláció, visszaállítás
test_strategy_advanced.py	35	ThresholdStrategy: stop-loss, flip, scale-in, cooldown, diagnosztika
Összesen	139	

5.3. táblázat. A fehérdoboz egységtesztek áttekintése.

2. Integrációs tesztek (feketedoboz)

Az egész pipeline-t tesztelik összefűzött komponenseken, valódi interfészek mögött.

Tesztfájl	Db	Mit ellenőriz
test_binance_feed.py	6	Binance feed üzenetfeldolgozás, újra-csatlakozás
test_integration_pipeline.py	9	Stratégia + kockázat + végrehajtás összefűzött pipeline
test_e2e_pipeline.py	1	Buy-Sell körforgalom teljes pipeline-on
test_e2e_integration.py	4	TradingEngine + EventBus, teljes kitöltési ciklus
Összesen	20	

5.4. táblázat. Az integrációs (feketedoboz) tesztesetek áttekintése.

3. Rendszertesztek (E2E forgatókönyvek)

Teljes, valós bekötésű forgatókönyveket hajtanak végre a rendszer egészén keresztül.

Tesztfájl	Db	Mit ellenőriz
<code>test_system_scenarios.py</code>	8	Nyereséges long/short kereskedés, több eszköz, kockázati blokkolás

5.5. táblázat. A rendszer-szintű (E2E) tesztesetek áttekintése.

4. Terhelési tesztek

Nagy adatmennyiség melletti viselkedést és szálbiztonságot mérnek.

Tesztfájl	Db	Mit ellenőriz
<code>test_load.py</code>	9	10 000 stratégia-predikció, 5 000 kockázatértékelés, szálbiztonság, throughput-mérés

5.6. táblázat. A terhelési tesztesetek áttekintése.

A tesztelés során szerzett tapasztalatok alapján több implementációs döntés módosult: az eredetileg egyoldalas `prob_up > 0.5` küszöb helyett aszimmetrikus `[0.45, 0.55]` neutral-sávot vezettem be (`test_signal_diagnostics.py` fedte fel a túlzott BUY-bias-t), és a díjszámítási hibát a `test_fee_fix.py` tárta fel. A terhelési tesztek megerősítik, hogy a stratégia és a kockázatkezelő réteg több szálon párhuzamosan futtatva is determinisztikus és versenyfeltétel-mentes eredményt ad.

5.5. A visszatesztelő keretrendszer

A visszatesztelő (`adapters/testing/backtest.py`) az összes kereskedési logikát ugyanazon `IBrokerGatewayPort` és `IMarketFeedPort` interfészekon keresztül futtatja, mint az élő rendszer. A visszateszteléshez szimulált eseménybusz (`InMemoryEventBus`) és determinisztikus időforrás (`FixedTimeSource`) kerül felhasználásra, így a tesztek reprodukálhatók.

A futtatás menete: a historikus OHLCV-adatot a rendszer betölti és normalizálja, offline betanítja a modellt, majd percenkénti szimulációs loop-ban végighalad a validációs perióduson: 0,1% taker és 0,1% maker díjakkal. Az eredményt tőkegörbe, Sharpe-mutató, maximális visszaesés és win-rate formájában naplózza.

5.6. Kockázatkezelés

A kockázatkezelési réteg (`core/domain/risk.py`) négy alapszabályt kényszerít ki. Pozícióméret-korlát: egyetlen nyitás sem haladhatja meg a tőke 10%-át.

Aszimmetrikus stop-loss: hosszú pozíciónál -2% , rövidnél $-1,5\%$ belépési ártól. Ha egyszerre 5-nél több szimbólum lenne nyitott, az újabb jelzés blokkolódik, és ha a napi veszteség meghaladja a 3% -ot, a motor az összes pozíciót zárja.

Ezentúl csak akkor adódik ki megbízás, ha a KCMamba K -gain értéke meghaladja a $K_{\min} = 0,15$ küszöböt, ami azt jelzi, hogy a modell elegendő bizalommal rendel előrejelzéshez az aktuális adatminőség alapján.

5.7. Élő kereskedés Binance-szal

Az éles kereskedési adapter (`adapters/infrastructure/binance_broker.py`) a Binance Futures REST és WebSocket végpontjait használja. Az árfolyam-frissítések az `IEventBusPort`-on keresztül jutnak el a `TradingEngine`-hez, az üzleti logika nem tud a Binance-specifikus adatformátumról, az adapter cserélhető.

A rendszer papírkereskedelmi (*paper trading*) módban is futtatható. Ebben az esetben a Binance-adapter le van cserélve a `adapters/testing/broker.py` szimulált végrehajtójára, így az algoritmus valódi tőke kockáztatása nélkül tesztelhető éles piaci adaton.

5.8. A rendszer futtatása

A teljes rendszer egyetlen belépési ponton, a `main.py` fájlban keresztül indítható:

```
# Visszatesztelési mód (nincs hálózati kapcsolat)
python main.py --mode backtest --symbol BTCUSDT --from 2024-01-01 --to 2024-12-31

# Papírkereskedési mód (Binance feed, szimulált végrehajtás)
python main.py --mode paper --symbol BTCUSDT ETHUSDT

# Éles kereskedési mód (Binance API-kulcs szükséges)
python main.py --mode live --symbol BTCUSDT
```

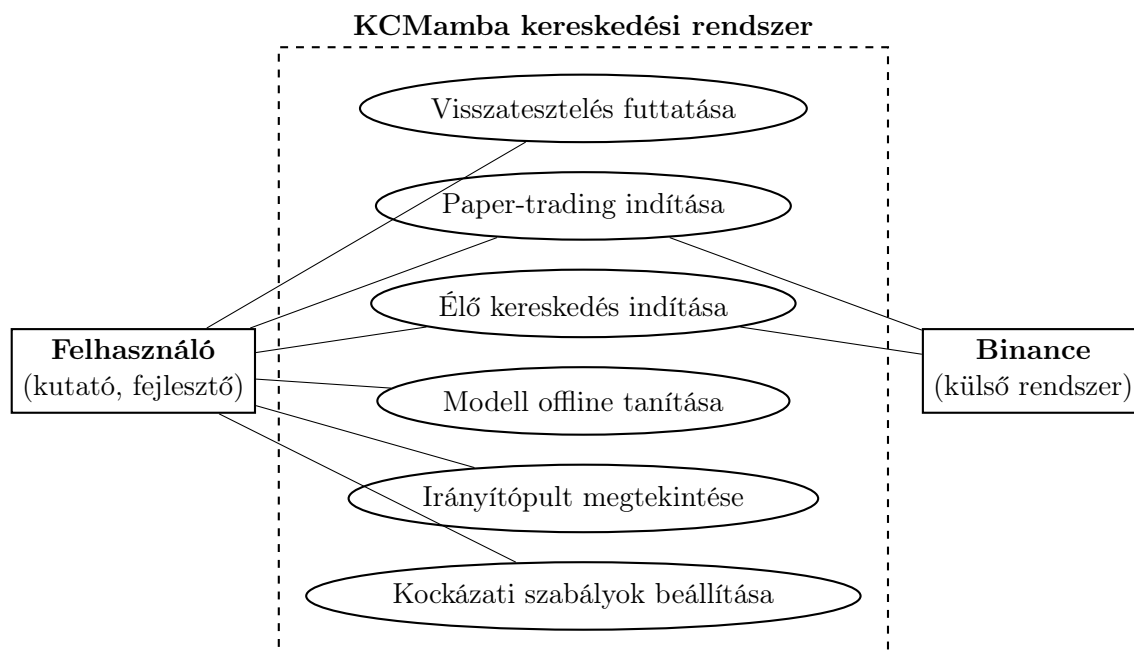
Az irányítópult automatikusan elindul, és a `http://localhost:8080` címen érhető el. Az API-kulcsok `.env` fájlból töltődnek be, sohasem kerülnek a forráskódba.

6. fejezet

Felhasználói dokumentáció

6.1. Használati esetek

A rendszer két szereplőt különböztet meg. A **Felhasználó** (kutató, fejlesztő) a kereskedési motort és a webes irányítópultot vezérli, míg a **Binance** külső rendszerként szolgáltat piaci adatot, és fogadja a megbízásokat. Az 6.1. ábra a támogatott használati eseteket foglalja össze.



6.1. ábra. A KCMamba rendszer használati esetei. A felhasználó hat fő művelethez fér hozzá; a Binance külső rendszerként a paper- és élő-kereskedési útvonalon kapcsolódik be (piaci adat, megbízás-végrehajtás).

6.2. A szoftver célja és célközönsége

A rendszer kriptovaluta idősorok automatizált elemzésére és kereskedési jelzések generálására készült, elsősorban BTCUSDT és ETHUSDT párokon. Célközönsége fejlesztők és kutatók, akik ML-alapú kereskedési stratégiákat szeretnének visszatesztelni vagy élőben futtatni.

6.3. Felhasznált módszerek áttekintése

A rendszer az árfolyam-előrejelzéshez a **KCMamba** architektúrát használja, amely a Mamba szelektív állapotter-modelle (SSM) egy differenciálható Kalman-szűrő réteget integrál: tanult Q_{net} , R_{net} és K_{net} alhálózatok időlépésenként becsülik a folyamat- és mérési zaj arányát, és ennek megfelelően adaptív K -gainnel szabályozzák, hogy a modell mennyire bízjon az aktuális megfigyelésben az előzmény-állapottal szemben (részletes leírás: 3. fejezet). A bemeneti OHLCV-gyertyákból csúszóablakos jellemzővektorok készülnek, amelyeket a modell 96 lépéses előzmény alapján 96 lépésre előre jelez. A predikciókból egy küszöb-alapú stratégia (**ThresholdStrategy**) generál BUY/SELL/HOLD jelzéseket, amelyeket egy kockázatkezelő réteg (pozícióméret-korlát, stop-loss, napi veszteség-limit) szűr, mielőtt a megbízás a Binance Futures REST végponton kiküldésre kerülne. Az architekturális döntések, a tanítási protokoll és a kísérleti értékelés részletei a 3. és 4. fejezetben találhatók.

6.4. Rendszerkövetelmények

Összetevő	Minimum	Ajánlott
Operációs rendszer	Linux/macOS/Windows 10+	Ubuntu 22.04 LTS
Python verziója	3.10	3.11
RAM	4 GB	8 GB
Lemezterület	2 GB	5 GB
GPU (opcionális)	NVIDIA ≥ 6 GB VRAM	NVIDIA ≥ 12 GB VRAM
Hálózat	n.a. (backtest módban)	Stabil internet (live)

6.1. táblázat. A rendszer minimális és ajánlott hardver/szoftvere.

6.5. Telepítési útmutató

1. Virtuális környezet létrehozása a projekt mappájában:

```
cd diplomamunkakod
python -m venv .env
# Windows (PowerShell -- egyszer kell futtatni):
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
.venv\Scripts\Activate.ps1
# Linux / macOS:
# source .venv/bin/activate
```

2. Függőségek telepítése (ML-csomagok opcionálisak):

```
pip install -r requirements.txt
pip install -r requirements-ml.txt # GPU-ra: CUDA-kompatibilis PyTorch
```

3. API-kulcsok beállítása (csak élő/paper módban szükséges):

```
cp .env.example .env
# Szerkeszd a .env fájlt: BINANCE_API_KEY, BINANCE_SECRET
```

6.6. Használati útmutató

Első indítás (paper-trading demó)

A rendszer alap-használati módja egy előtérben futó paper-trading demó. A `main.py` szkript végrehajtása egyszerre három háttérszolgáltatást indít: egy offline tanítófolyamot, a Binance WebSocket-feed-et és a webes irányítópultot.

```
source .venv/bin/activate
python main.py
```

Sikeres indítás esetén a konzol az alábbi blokkokat írja ki: a `core` modulok betöltése, az adapterek inicializálása, a historikus adatcsomag letöltése (a `warmup_start_str` szerint, alapértelmezetten 3 napra visszamenőleg), majd a `[BOOT] TradingEngine started` szalag, amely után az irányítópult a `http://127.0.0.1:8080` címen érhető el.

Dashboard-nézetek

Az irányítópult felső navigációs sávja nyolc nézetre tagolódik, három kategóriába rendezve:

Kategória	Oldal	Tartalom
Pénzügyi	/	Tőkegörbe, nyitott pozíciók, élő ár-chart
Pénzügyi	/signals.html	Generált jelzések listája, kockázati státusszal
Pénzügyi	/model.html	Modellverzió, kalibráció, diagnosztikai metrikák
Pénzügyi	/training.html	Offline tanítás előrehaladása, loss-görbe
Benchmark	/benchmark.html	Szintetikus zajos teszt eredménye
Benchmark	/ltsf_benchmark.html	LTSF (Exchange, ETTh) benchmark-eredmények
KCA elemzés	/kla_kalman.html	Kalman-paraméterek (K , A , R) valós idejű görbéi
KCA elemzés	/complexity.html	Paraméterszám $\times$ teljesítmény heatmap

6.2. táblázat. Az irányítópult nézeteinek szerepköre.

A jobb alsó sarokban elhelyezett ? gombra kattintva oldal-specifikus súgóablak nyílik, amely röviden elmagyarázza az adott nézetet. A lapok közti átjárás a felső menüsávból történik, a böngésző *vissza*-gombja is használható.

Használati minták

A rendszer négy tipikus forgatókönyvben futtatható. A paper-trading demó (alapértelmezett) Binance-adaton szimulált végrehajtással dolgozik. A visszateszt-mód ugyanazt a pipeline-t futtatja historikus adatokon, Binance kapcsolat nélkül, `adapters/testing/backtest.py` segítségével. Az eredményeit a konzolra és CSV-be írja. Az offline tanítás a teljes dataset-et (alapértelmezetten 3 nap BTCUSDT 5 perces gyertyái) egy menetben dolgozza fel, majd a súlyokat `*.pt` fájlba menti. A *csak-dashboard* mód végül akkor hasznos, ha egy korábbi futás eredményeit akarjuk áttekinteni: ekkor a Binance-feed kikapcsolható, és a dashboard offline módban is böngészhető.

Futásidei naplózás és leállítás

A `stdout` a rendes állapotüzeneteket tartalmazza (`[BOOT]`, `[SIGNAL]`, `[FILL]`, `[DASHBOARD]`), a `stderr` pedig a figyelmeztetéseket és hibákat (`[WARN]`, `[FATAL ERROR]`). A két csatorna külön is naplózható:

```
nohup python main.py > run_log.txt 2> run_err.txt &
```

A futtatás **Ctrl+C**-vel bármikor biztonságosan megszakítható. A **finally** ág garantálja, hogy (i) a WebSocket-kapcsolat lezárul, (ii) az in-memory esemény-pufferek kiürülnek, (iii) a HTTP-szerver lezárja az aktív kapcsolatokat, és (iv) a végleges állapot (nyitott pozíciók, tőkeállás, utolsó jelzés) kiírásra kerül a **FINAL DASHBOARD SNAPSHOT** fejléc alatt.

Biztonsági megfontolások

A demó alapértelmezés szerint *paper-trading* módban fut, valódi Binance-rendelést nem küld. Ha mégis éles kulcs kerül a `.env` fájlba, a Binance-oldalon ajánlott (a) csak olvasási és Futures-kereskedési jogosultságot adni, és (b) IP-whitelisttel megszorítani a kulcs használatát. Az irányítópult alapértelmezetten a `0.0.0.0:8080` címen hallgat. nyilvános IP-n csak külső **reverse proxy** (nginx, traefik) mögött ajánlott közzétenni.

6.7. Hibaelhárítás

`ModuleNotFoundError: mamba_ssm`

Telepítsd a `requirements-ml.txt`-ben szereplő CUDA-kompatibilis csomagjait, vagy futtasd CPU-n: `python main.py -device cpu`.

Binance API 403/401

Ellenőrizd a `.env` fájl kulcsait és az IP-whitelist beállításait a Binance kereskedési felületén.

WebSocket kapcsolódási hiba

A tűzfal blokkolhatja a Binance `wss://` végpontját (9443 port). Szükség esetén proxyt kell beállítani.

6.8. Futásidei üzenetek és hibaelhárítás

Az alábbi táblázat a leggyakoribb futásidei üzeneteket és a javasolt teendőket foglalja össze.

Üzenet / hibaüzenet	Súlyosság	Teendő
[WARN] Offline training failed	Figyelmeztetés	A rendszer az előző modellel folytatja; ellenőrizd az internet-kapcsolatot és az adatok elérhetőségét.
[WebSocket] Connecting...	Tájékoztató	Automatikus újracsatlakozás; ha 5 kísérlet után sem sikerül, ellenőrizd a tűzfalat (9443 port).
ModuleNotFoundError: mamba_ssm	Hiba	Telepítsd a <code>requirements-ml.txt</code> csomagjait, vagy adj meg <code>-device cpu</code> kapcsolót.
Binance API 401/403	Hiba	Érvénytelen API-kulcs; frissítsd a <code>.env</code> fájlt és ellenőrizd az IP-whitelistet.
Not enough data (N rows)	Figyelmeztetés	Növeld a <code>training_start_str</code> ablakot, vagy várj, amíg több zárt gyertya gyűlik össze.
[FATAL ERROR] ...	Kritikus	Verem-nyomat a <code>stderr</code> -en; küldd el a teljes naplót a fejlesztőnek.

6.3. táblázat. A leggyakoribb futásidei üzenetek és a javasolt hibaelhárítási lépések.

7. fejezet

Összegzés

7.1. Főbb eredmények

A KCMamba architektúra differenciálható Kalman-szűrési mechanizmust integrál a Mamba SSM-be. A három adatkészleten végzett kísérletekből az alábbi következtetések vonhatók le:

1. Zajrobustság: $\sigma = 0 \rightarrow \sigma = 5$ zajszint-növelésnél a KCMamba átlagos MSE-degradációja +1 528%, a Fair Mambáé +28 234%, ami 18,5 $\times$ -os különbség. Exchange/ $s = 16$ -nál a különbség öt nagyságrend: +66% vs +152 350%.
2. Hosszú szekvencia (4.5. fejezet): az új mérések ($T \in \{96, 192, 336, 512\}$, ETTm1 + Exchange) a KCMamba erősségét hosszabb kontextusnál is igazolják.
 - $T = 336, \sigma = 1$: KCMamba MSE 0,502 vs LSTM 0,735 (**-31,7%**, a legnagyobb LSTM-mel szembeni arány).
 - $T = 512, \sigma = 3$ (Exchange): KCMamba 0,773, Fair Mamba 2,121 (**-63,6%**, a legnagyobb Fair Mambával szembeni arány).
 - ETTm1-en $\sigma = 3$ -nál a KCMamba MSE-je egyedüli modellként monoton csökken a szekvenciahossz növekedésével ($0,673 \rightarrow 0,466, -31\%, T: 96 \rightarrow 512$).
3. Stride $\times$ szekvenciahossz (4.6. fejezet): az Exchange adatkészleten $s \in \{1, 16\}$ és $T \in \{192, 336, 512\}$ kombinációkban mérve két mintázat rajzolódott ki.

$s=1$, $\sigma=3$ -nál a KCMamba előnye az LSTM-mel szemben monoton nő a szekvenciahosszal ($T=512$ -nél $+35\%$: 0,836 vs 1,292). $s=16$, $\sigma \geq 3$ -nál a Fair Mamba katasztrofálisan összeomlik (8,1–8,5 MSE), ami $7\text{--}9\times$ -es stabilitásbeli különbség a KCMamba-hoz képest, amely 1,1–1,3 között marad.

4. Kalman-gain adaptáció: a modell által az adatokból megtanult K -érték a zajszint emelkedésével monoton csökken ($0.706 \rightarrow 0.073$), ami konzisztens az explicit Kalman-egyenletekből származtatható várakozásokkal. Ez megerősíti, hogy az architektúráis prior érdemben irányítja a tanulási folyamatot.
5. Kevés adat: $s = 16$ stride esetén a Kalman-struktúra implicit regularizáló hatást fejt ki, ami ellensúlyozza az adathiányt.

7.2. Korlátok

A KCMamba nem minden konfigurációban jobb, és a kísérleti keretrendszer több olyan egyszerűsítést tartalmaz, amelyek a következtetések általánosíthatóságát érdemben korlátozzák.

$\sigma = 0$, $s = 16$ esetén a Fair Mamba szinte memorizál, és jóval kisebb MSE-t ér el. Ez arra utal, hogy zajtalan környezetben az explicit Kalman-struktúra felesleges terhet jelent: a mindig aktív szűrési tag enyhe aláilleszkedést okoz, és az architektúra nem képes teljesen kikapcsolni a szűrést.

ETTh1/ $s = 1$ -nél az LSTM teljesít a legjobban, mivel a kapuzási mechanizmusa kihasználja a nagy adatmennyiséget. Az LSTM forget/input gate párosa hasonló adaptív szűrést valósít meg, de rugalmasabb, mert nem kötött a $K = Q/(Q + R)$ struktúrához. Ez nem a KCMamba kudarca, hanem annak illusztrációja, hogy adatgazdag rezsimben az induktív prior haszna csökken.

Mindössze 5 epoch futott, egyetlen véletlenszerű maggal (seed), így a bemutatott eredmények konfidencia-intervallum nélküli pontbecslések.

A zajmodell kizárólag izotróp Gauss-zaj ($\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$). A valós alkalmazásokban a zaj általában heteroszkedasztikus, időben korrelált, esetenként nem-Gauss eloszlású, és sokszor strukturális jellegű (szenzorkiesés, hiányzó érték). Az itt mért robusztusság ezekre a zajformákra nem általánosítható automatikusan.

A skalár Q_t , R_t becslés azt feltételezi, hogy a zajkomponensek csatornánként függetlenek. Diagonális vagy teljes kovarianciamátrix becslése gazdagabb zajmodellt adna, de érzékelhetően növelné a paraméterszámot.

A hiperparaméter-érzékenység átfogó vizsgálata (rejtett dimenzió, fejek száma, rétegszám, kernelméret, tanulási sebesség) nem történt meg. A bemutatott konfiguráció ésszerű alapbeállítás, de nem garantáltan Pareto-optimális.

7.3. Jövőbeli irányok

Az $s \in \{1, 4, 8, 16, 32, 64\}$ értékek szisztematikus végigpásztázása megmutatná, melyik adatszegénységi küszöbnél jelenik meg először a Kalman-struktúra szabályozó hatása. A szimulált Gauss-zaj szenzorkiesésekkel, hiányzó értékekkel és kilógó megfigyelésekkel való felváltása az architektúrát a valós alkalmazási feltételekhez közelítené. Egy meta-hálózat alapú zajszint-becslés lehetővé tenné, hogy σ tanított paraméterré váljon, és a modell ismeretlen zajú környezetekhez automatikusan alkalmazkodjon. A skalár Q_t , R_t diagonális Cholesky-faktorizációval való felváltása gazdagabb bizonytalanság-becslést és jobb kalibrációt eredményezhet.

Köszönetnyilvánítás

A szerző köszönetét fejezi ki a témavezetőnek a szakmai iránymutatásért, valamint az ELTE Informatikai Karának a kutatási infrastruktúra biztosításáért. Külön köszönet illeti az idősor-előrejelzési közösséget a nyílt adatkészletekért (ETTh1, ETTh2, Exchange-Rate), amelyek nélkül az empirikus értékelés nem lett volna lehetséges.

Irodalomjegyzék

- [1] Albert Gu, Karan Goel és Christopher Ré. „Efficiently Modeling Long Sequences with Structured State Spaces”. *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://openreview.net/forum?id=uYLFoz1v1AC>.
- [2] Albert Gu és Tri Dao. „Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. *First Conference on Language Modeling (COLM)*. 2024. URL: <https://openreview.net/forum?id=tEYskw1VY2>.
- [3] Sepp Hochreiter és Jürgen Schmidhuber. „Long Short-Term Memory”. *Neural Computation* 9.8 (1997), 1735–1780. old. DOI: 10.1162/neco.1997.9.8.1735.
- [4] Rudolf E. Kalman. „A New Approach to Linear Filtering and Prediction Problems”. *Transactions of the ASME – Journal of Basic Engineering* 82.1 (1960), 35–45. old. DOI: 10.1115/1.3662552.
- [5] Rahul G. Krishnan, Uri Shalit és David Sontag. „Deep Kalman Filters”. *arXiv preprint arXiv:1511.05121* (2015). URL: <https://arxiv.org/abs/1511.05121>.
- [6] Marco Fraccaro és tsai. „A Disentangled Recognition and Nonlinear Dynamics Model for Unsupervised Learning”. *Advances in Neural Information Processing Systems (NeurIPS)*. 30. köt. 2017. URL: <https://arxiv.org/abs/1710.05741>.
- [7] Haoyi Zhou és tsai. „Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting”. *AAAI Conference on Artificial Intelligence*. 35. köt. 12. 2021, 11106–11115. old. DOI: 10.1609/aaai.v35i12.17325.
- [8] Haixu Wu és tsai. „Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting”. *Advances in Neural Information Processing Systems (NeurIPS)*. 34. köt. 2021, 22419–22430. old.

- URL: <https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html>.
- [9] Ailing Zeng és tsai. „Are Transformers Effective for Time Series Forecasting?": *AAAI Conference on Artificial Intelligence*. 37. köt. 9. 2023, 11121–11128. old. DOI: 10.1609/aaai.v37i9.26317.
- [10] Guokun Lai és tsai. „Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks". *International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2018, 95–104. old. DOI: 10.1145/3209978.3210006.
- [11] Yuqi Nie és tsai. „A Time Series is Worth 64 Words: Long-term Forecasting with Transformers". *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://openreview.net/forum?id=Jbdc0vT0col>.
- [12] Guy E. Blelloch. *Prefix Sums and Their Applications*. Techn. jel. CMU-CS-90-190. School of Computer Science, Carnegie Mellon University, 1990. URL: <https://www.cs.cmu.edu/~guyb/papers/Ble90.pdf>.
- [13] Ashish Vaswani és tsai. „Attention Is All You Need". *Advances in Neural Information Processing Systems (NeurIPS)*. 30. köt. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.

Ábrák jegyzéke

- 3.1. A KCMamba blokk teljes adatfolyam-diagramja. A sárga blokkok $(Q_{\text{net}}, R_{\text{net}}, K_{\text{net}})$ tanítják a Kalman-statisztikákat, a lila blokkban történik a párhuzamos prefix-scan GPU-n $O(\log T)$ szinkronizációval. A zöld blokkok a bemeneti és kimeneti projekciókat jelölik. 17
- 3.2. A Blelloch-féle párhuzamos prefix-scan vizualizációja 8 elemű szekvencián. Minden lépésben a *step*-nyi távolságú két elem kombinálódik. $\log_2 T = 3$ lépés után minden h_t tartalmazza az előzmény-összesítést. Ez teszi lehetővé a szekvenciális állapotfrissítés hatékony GPU-s párhuzamosítását. 21
- 3.3. A KCMamba verem (stack) felépítése. A bemenettől a kimenetig az input projekció, 2 KC-blokk LayerNorm rétegekkel, output normalizáció, majd cross-attention réteg követi egymást a lassú ($s = 4$) időskála integrálásához. 23
- 3.4. Szintetikus jel szűrése: **vanilla SSM** (standard Mamba, rögzített A) vs. **KCMamba** (adaptív K -gain) közepes ($\sigma = 2$) és erős ($\sigma = 5$) zajnál. *Felső sorok*: az előrejelzett jel a valódihoz képest. A KCMamba szorosabban követ zajban is. *Alsó sorok*: a tanult K -értékek időbeli változása. A vanilla SSM A -ja rögzített, a KCMamba A -ja adaptívan nő zajjal. 24
- 3.5. **Bal**: Impulzus-válasz függvény: az állapot exponenciális csökkenése $h_k = A^k$ a k késleltetés függvényében. A KCMamba A -értéke adaptívan nő a zajszinttel ($0.294 \rightarrow 0.926$), ezzel a memória-horizont is növekszik. **Jobb**: Effektív memória-horizont (az a lépésszám, ameddig $A^k > 5\%$) a zajszint függvényében. Az LSTM és vanilla SSM rögzített horizonttal rendelkeznek, a KCMamba zajban automatikusan hosszabb memóriát tart fenn. 24

- 3.6. A KCMamba alapú kereskedési rendszer hexagonális architektúrájának áttekintése. A **Core** csomag tartalmazza a TradingEngine-t (pipeline: állapotbecslés $\rightarrow$ predikció $\rightarrow$ szignál $\rightarrow$ kockázat $\rightarrow$ végrehajtás), a domain modelleket és az alkalmazásportokat. A külső adapterek (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) kizárólag az interfészeken (IEventBusPort, IBrokerGatewayPort stb.) keresztül kommunikálnak a Core-ral. . . . 27
- 4.1. **Bal:** MSE a zajszint függvényében, Exchange/ $s = 16$ konfiguráció. A Fair Mamba a legjobb zajmentes MSE-t adja ($\sigma = 0$), de jelentős degradációt mutat $\sigma = 5$ -nél. A KCMamba degradációja $+66\%$ -nál stabilizálódik. **Jobb:** A KCMamba Kalman-gain (K), felejtési faktor ($A = 1 - K$) és zajbecslés (R) automatikus adaptációja a zajszint növekedésével. Magas zajnál K meredék esést mutat ($0,706 \rightarrow 0,073$). 31
- 4.2. MSE hő térkép mind a négy modellre (ARIMA, LSTM, Fair Mamba, KCMamba), minden konfigurációra és zajszintre. A sötétpiros mezők magas hibát jeleznek, $MSE > 4$. Az ARIMA és a Fair Mamba a $s=16$ konfigurációkban összességében $\sigma \geq 3$ -nál csődöt vallanak, míg a KCMamba végig stabil marad. 32
- 4.3. MSE-degradáció logaritmikus skálán ($\sigma = 0 \rightarrow \sigma = 5$) az összes konfigurációban. A Fair Mamba három esetben erős degradációt mutat ($> 10\,000\%$). A KCMamba (zöld) és az LSTM (kék) lényegesen stabilabbak. A kiemelten jelölt mező az Exchange/ $s = 16$ esetet jelöli. 33
- 4.4. Zajrobustussági összehasonlítás mind a négy modellre: MSE a zajszint (σ) függvényében, logaritmikus skálán. Az ARIMA (narancssárga, szaggatott) $s=16$ -nál katasztrofálisan összeomlik ($\sigma=5$ -nél $MSE > 15$). A Fair Mamba szintén instabillá válik $s=16$ -nál. A KCMamba és az LSTM végig mérsékelt tartományban maradnak. 34
- 4.5. Előrejelzés-vizualizáció három zajszinten (ETTh-típusú jel, $T = 96$ lookback, $H = 96$ horizon). Fekete = valós jel, zöld = KCMamba, narancssárga = Fair Mamba, kék = LSTM. **Bal:** $\sigma = 0$ -nál mindkét Mamba-modell pontosan követi a jelet. **Közép/Jobb:** A zaj növekedésével a KCMamba sima marad, a Fair Mamba instabillá válik, az LSTM lassan konvergál. 34

4.6. Bal: Paraméterszám vs. MSE ($\sigma=3$ átlag). A KCMamba egyszerre a legkisebb paraméterszámú és a legjobb MSE-jű, optimális Pareto-pozícióban van. Jobb: Epoch-tanítási idők (másodpercben). A PyTorch implementáció KCMamba-t $\approx 40\%$ -kal lassítja, de Triton-kernellel gyorsabb lesz az LSTM-nél.	35
4.7. Bal: Áteresztőképesség (minta/s) $T = 96$ -nál, kötegméret = 32, RTX 3060-on. A cuDNN-optimalizált LSTM a referencia; a natív PyTorch szelektív scan $3,4\times$ lassabb, egy Triton-kernelre írt KCMamba viszont LSTM-szintű sebességet ér el. Jobb: LSTM-hez normált relatív sebesség. A Triton-kernel rövid szekvenciákra szükséges, de nem elégséges optimalizáció. $T \geq 2000$ -nél a scan $O(\log T)$ mélysége várhatóan $2\text{--}10\times$ -es előnyt ad a szelektív SSM-ek irodalmában [2].	36
4.8. MSE a szekvenciahossz (T) függvényében $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten, ETTm1 (felső sor) és Exchange (alsó sor) adatkészleteken. A KCMamba az egyetlen modell, amely mindkét adatkészleten és mindkét zajszinten javuló trendet mutat a szekvenciahossz növekedésével.	38
4.9. MSE a szekvenciahossz függvényében sűrű ($s=1$, felső sor) és ritka ($s=16$, alsó sor) ablakozásnál, $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten az Exchange adatkészleten. Az $s=16$ panelokon a tengely 4,0-nál el van vágva (a Fair Mamba MSE-je 8,1–8,5-re ugrik). A KCMamba minden konfigurációban stabil. A Fair Mamba $s=16$ -nál $\sigma \geq 3$ esetén katasztrofikusan összeomlik.	40
5.1. Az irányítópult főoldala. A felső sávon a tőke-görbe, alatta a nyitott pozíciók és a legutóbbi jelzések láthatók.	43
5.2. A kereskedési jelzések nézete. Minden sor egy generált jelzést ábrázol a jelzés erősségével és a döntéshozó modell állapotával.	43
5.3. A modell diagnosztikai nézete. A Chart.js-alapú panel valós időben frissül az előrejelzésekkel és eltérésekkel.	44
5.4. Az offline tanítás irányítópultja. A tanítás teljes menete nyomon követhető, beleértve a korai leállás feltételét is.	45

- 5.5. A benchmark összehasonlítási nézet. Az interaktív szűrők lehetővé teszik az adatkészlet, zajszint és stride dinamikus változtatását. . . . 45
- 6.1. A KCMamba rendszer használati esetei. A felhasználó hat fő művelethez fér hozzá; a Binance külső rendszerként a paper- és élő-kereskedési útvonalon kapcsolódik be (piaci adat, megbízás-végrehajtás). 49